

Filippo Valiante Filho

Introdução a **sistemas operacionais**

Dados Internacionais de Catalogação na Publicação (CIP)
(Claudia Santos Costa – CRB 8ª/9050)

Valiante Filho, Filippo

Introdução a sistemas operacionais / Filippo Valiante Filho. –
São Paulo : Editora Senac São Paulo, 2025. – (Série Universitária)

Bibliografia.

e-ISBN 978-85-396-5590-8 (ePub/2025)

e-ISBN 978-85-396-5591-5 (PDF/2025)

1. Sistemas operacionais (Computadores). I. Título. II. Série.

25-2483c

CDD – 005.43
BISAC COM046000

Índice para catálogo sistemático:

1. Sistemas operacionais (Computadores) 005.43

INTRODUÇÃO A SISTEMAS OPERACIONAIS

Filippo Valiante Filho





Administração Regional do Senac no Estado de São Paulo

Presidente do Conselho Regional

Abram Szajman

Diretor do Departamento Regional

Luiz Francisco de A. Salgado

Superintendente Universitário e de Desenvolvimento

Luiz Carlos Dourado

Editora Senac São Paulo

Conselho Editorial

Luiz Francisco de A. Salgado

Luiz Carlos Dourado

Darcio Sayad Maia

Lucila Mara Sbrana Sciotti

Luís Américo Tousi Botelho

Gerente/Publisher

Luís Américo Tousi Botelho

Coordenação Editorial

Verônica Marques Pirani

Prospecção

Andreza Fernandes dos Passos de Paula

Dolores Crisci Manzano

Paloma Marques

Administrativo

Marina P. Alves

Comercial

Aldair Novais Pereira

Comunicação e Eventos

Tania Mayumi Doyama Natal

Coordenação de Arte

Antonio Carlos De Angelis

Coordenação de Revisão de Texto

Marcelo Nardeli

Acompanhamento Pedagógico

Otacília da Paz Pereira

Designer Educacional

Janaina Nascimento Menezes da Silva

Revisão Técnica

Alexandre dos Santos Mignon

Preparação e Revisão de Texto

Mariana Faria

Projeto Gráfico

Alexandre Lemes da Silva

Emília Corrêa Abreu

Capa

Antonio Carlos De Angelis

Editoração Eletrônica e Ilustrações

Alyne Lourenço de Souza

Alissa Queiroz

Imagens

Adobe Stock Photos

Proibida a reprodução sem autorização expressa.

Todos os direitos desta edição reservados à

Editora Senac São Paulo

Av. Engenheiro Eusébio Stevaux, 823 – Prédio Editora

Jurubatuba – CEP 04696-000 – São Paulo – SP

Tel. (11) 2187 4450

editora@sp.senac.br

<https://www.editorasenacsp.com.br>

© Editora Senac São Paulo, 2025

Sumário

Capítulo 1 Fundamentos dos sistemas operacionais, 7

- 1 Sistema operacional, 8
 - 2 Breve histórico das plataformas computacionais e dos sistemas operacionais, 10
 - 3 Tipos de sistema operacional, 16
 - 4 Preparando um sistema para a prática ao longo do curso, 18
- Considerações finais, 19
- Referências, 19

Capítulo 2 Arquitetura dos sistemas operacionais, 21

- 1 O núcleo (kernel), 22
 - 2 O espaço do usuário, 24
 - 3 A interface com o usuário, 24
 - 4 A inicialização do sistema, ou como se levantar segurando as alças de suas botas!, 27
 - 5 Prática: comandos básicos do Linux e do Unix, 28
- Considerações finais, 30
- Referências, 30

Capítulo 3 Arquitetura de computadores: uma visão do suporte ao sistema operacional, 31

- 1 Atuação do sistema operacional sobre o hardware, 34
 - 2 Concorrência, interrupções e exceções, 36
 - 3 Prática: visualizando informações sobre o hardware e o sistema operacional, 39
- Considerações finais, 40
- Referências, 41

Capítulo 4 Processos e threads: porque os computadores existem para executar programas, 43

- 1 Usuário privilegiado: porque alguém precisa assumir a responsabilidade, 47
 - 2 Threads, 48
 - 3 Canalização e redirecionamento de processos, 49
 - 4 Prática: o usuário administrador, 50
- Considerações finais, 51
- Referências, 51

Capítulo 5 Gerenciamento de processos, 53

- 1 Os algoritmos de escalonamento dos sistemas operacionais atuais, 56
 - 2 Deadlocks: quando as coisas dão errado, 58
 - 3 Prática: monitoramento e gerenciamento de processos, 59
- Considerações finais, 60
- Referências, 61

Capítulo 6 Gerenciamento de memória, 63

- 1 Memória virtual, 65
 - 2 Quando não há memória principal suficiente, 68
 - 3 Prática: monitoramento da memória e gerenciamento de pacotes, 69
- Considerações finais, 71
- Referências, 72

Capítulo 7

Gerenciamento de sistema de arquivos, 73

1 Prática: manipulando o sistema de arquivos, 78

Considerações finais, 82

Referências, 82

Capítulo 8

Gerenciamento de E/S, 83

1 Quando um sistema se conecta com outros, 87

2 Prática: scripts, 89

Considerações finais, 92

Referências, 93

Sobre o autor, 95

Fundamentos dos sistemas operacionais

É muito provável que você esteja lendo este texto em um equipamento computacional, como um notebook, smartphone ou tablet. Caso esteja lendo em uma versão impressa, foi necessário um computador para realizar a impressão. Aliás, a própria impressora é um computador. Nesses computadores, foram utilizados diversos programas, ou aplicações, o que, em inglês, é chamado software. São exemplos de aplicações os editores de texto, planilhas eletrônicas, navegadores, visualizadores e editores de imagem e vídeo, e, por que não, alguns jogos para as horas de folga depois dos estudos.

Normalmente, em um equipamento computacional, são usadas diversas aplicações com diferentes finalidades. E o uso do termo “equipamento” é para lembrar que se trata disto: máquinas feitas com circuitos eletrônicos digitais, ou, em inglês, hardware. Para que haja essa flexibilidade de um computador executar diversas aplicações, alternar de uma para outra, viabilizar que uma se comunique com a outra no mesmo computador e até em outros computadores através de uma rede local ou da internet, é necessário que haja algum sistema de controle. Ou seja, algo que coordene os diversos componentes do hardware e o software. Esse é o papel de um sistema operacional.

1 Sistema operacional

Um usuário final, do mais leigo ao mais avançado, acaba considerando o sistema operacional o próprio computador. Frases como “meu computador é Windows”, ou “meu celular é Android” são ditas comumente por usuários leigos e, na verdade, fazem muito sentido. Muitas aplicações só podem ser executadas em um sistema operacional específico. Outras, estão disponíveis para vários sistemas operacionais, mas precisam ser instaladas sempre na versão de cada sistema. Um desenvolvedor vai fazer seu trabalho pensando no sistema operacional que executará a aplicação. E isso nos leva aos usuários mais avançados.

O sistema operacional permite que um desenvolvedor use uma instrução (método ou função na linguagem técnica) para ler o que o usuário digitou no teclado, para escrever uma mensagem na tela, para ler ou gravar um arquivo, imprimir e assim por diante. Ele não precisa descrever nos mínimos detalhes como fazê-lo a não ser que realmente seja esse o objetivo. Um desenvolvedor não precisa saber se o arquivo está sendo lido ou gravado em um pen drive, cartão de memória, disco rígido (HD) ou disco de estado sólido (SSD), ele vai usar a instrução adequada e irá funcionar. Obviamente não é mágica, e sim os serviços que estão

sendo fornecidos pelo sistema operacional. Outro detalhe importante é que o sistema operacional também é um software.

É possível responder à pergunta “o que é um sistema operacional?” de muitas maneiras. Para Tanenbaum e Bos (2024), é um software que visa “fornecer a programadores de aplicativos (e a programas de aplicativos, claro) um conjunto de recursos abstratos limpo em vez de recursos confusos de hardware e gerenciar esses recursos de hardware”. Maziero (2019) apresenta que

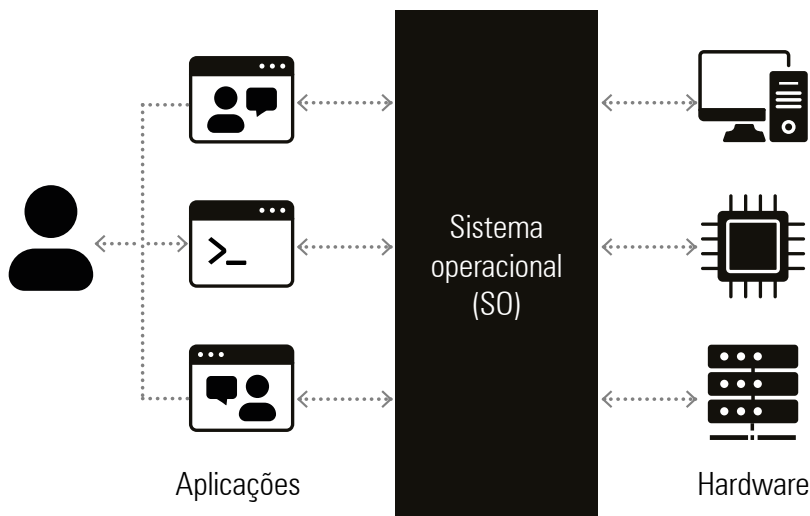
o sistema operacional é uma camada de software que opera entre o hardware e os programas aplicativos voltados ao usuário final. Trata-se de uma estrutura de software ampla, muitas vezes complexa, que incorpora aspectos de baixo nível (como drivers de dispositivos e gerência de memória física) e de alto nível (como programas utilitários e a própria interface gráfica). [...] os objetivos básicos de um sistema operacional podem ser sintetizados em duas palavras-chave: “abstração” e “gerência” (Maziero, 2019).

Por fim, os professores Machado e Maia (2013) definem que um sistema operacional

é apenas um conjunto de rotinas executado pelo processador, de forma semelhante aos programas dos usuários. Sua principal função é controlar o funcionamento de um computador, gerenciando a utilização e o compartilhamento dos seus diversos recursos, como processadores, memórias e dispositivos de entrada e saída (Machado; Maia, 2013).

Observe que as definições são bastante similares. Com isso, quero convidar você estudante a elaborar sua própria definição de sistema operacional! A seguir, para complementar essa definição, a figura 1 ilustra o posicionamento de um sistema operacional.

Figura 1 – O posicionamento de um sistema operacional



Nesse sentido, se o usuário pode digitar no teclado ou clicar com o mouse, por que se diz que ele não acessa diretamente o hardware? O que ocorre é que ao se pressionar uma tecla, clicar ou movimentar um mouse, um mecanismo de hardware faz com que o sistema operacional seja acionado. Ele então identifica o que aconteceu e encaminha para a aplicação correspondente. O sistema operacional se utiliza de alguns recursos especiais de hardware que serão estudados mais adiante.

Neste capítulo, abordaremos a evolução dos sistemas operacionais e os diferentes tipos de sistema operacional existentes. Daqui em diante, utilizarei a abreviação "SO" para me referir a sistema operacional. Em inglês, usa-se "OS", referente a operating system, como em iOS.

2 Breve histórico das plataformas computacionais e dos sistemas operacionais

O surgimento dos computadores eletrônicos digitais ocorreu no contexto da Segunda Guerra Mundial, sendo que os primeiros computado-

res passaram a ser comercializados logo ao fim desse período, mas basicamente para governos e empresas muito grandes, além dos computadores que começaram a ser desenvolvidos em universidades. Esses primeiros computadores executavam um programa por vez, que eram carregados por meio de cartões perfurados, e não possuíam SO. Na década de 1950, surgiram as linguagens de programação de alto nível e ferramentas como compiladores. Surgiram fitas perfuradas e posteriormente as fitas magnéticas passaram a ser usadas nos computadores. Os programas passaram a ser executados em sequência, um atrás do outro, ou “em lote” (em inglês, *batch*).

Na década de 1960, esses computadores de grande porte, ou mainframes, ganham mais periféricos e se tornam mais rápidos. Surgem os discos rígidos e um conjunto de teclado e monitor, que viabiliza a operação do computador, chamado “terminal”. Os processadores evoluem e incorporam recursos como interrupções. As linguagens de programação de alto nível ficam mais sofisticadas e disseminadas e, com o terminal, levam a uma distinção dos profissionais que trabalham com os computadores entre programadores e operadores. Para viabilizar a utilização de terminais e outros periféricos e possibilitar que mais de um programa fosse carregado na memória (multiprogramação), foi necessário o desenvolvimento de um programa de gerenciamento e controle para o computador e, assim, surgem os primeiros sistemas operacionais.

A multiprogramação, inicialmente, não envolvia que mais de um programa fosse executado simultaneamente, o que só é possível efetivamente com mais de um processador. Ela se limitava a permitir que mais de um programa estivesse carregado na memória do computador ao mesmo tempo. Dessa forma, um programa A pode ser executado, isto é, ocupar o processador, e quando ele precisa acessar um periférico, como escrever um arquivo em um disco, um programa B passa a ser executado.

No fim da década de 1960, Ken Thompson e Dennis Ritchie, trabalhando no Bell Labs, criaram o Unix (Tanenbaum; Bos, 2024), que, com seus derivados, constitui a família mais importante de sistemas operacionais.

A década de 1960 testemunhou também o surgimento do minicomputador, o que possibilitou que mais organizações pudessem ter acesso a computadores e, na virada para a década de 1970, o surgimento da Arpanet, que, anos depois, se tornaria a internet. Entre outros avanços, na década de 1970, são criados os circuitos integrados microprocessadores e surgem os primeiros computadores com múltiplos processadores. Também surgem os primeiros microcomputadores, impulsionados pelos primeiros microprocessadores, mas eles ainda não tinham larga utilização comercial. A consultoria Internet Data Corporation (IDC) sintetiza essas primeiras décadas da computação como a “primeira plataforma computacional”.

Já no início da década de 1980, a IBM lança sua versão do microcomputador, ou computador pessoal, chamada Personal Computer, o IBM-PC. O equipamento utilizava microprocessadores da Intel, derivado do modelo 8086 – por isso essa arquitetura computacional é até hoje chamada de x86. Os computadores se tornaram muito mais acessíveis, chegando a empresas menores e escolas e até mesmo às casas das pessoas. Poucos anos depois, a Apple lançaria o Macintosh, redefinindo os padrões de facilidade de uso com interfaces gráficas e amigáveis e o uso de mouse, ao contrário das interfaces meramente textuais em uso até então. Há muito mais usuários que agora não são especializados em computação. Há muito mais programas que focam esse usuário final e precisam ser mais intuitivos. Também há mais linguagens e ferramentas de programação e, obviamente, sistemas operacionais. Nessa década, surgem o sistema operacional de disco (DOS), o Windows, que, na época, ainda era uma interface gráfica amigável para o DOS, e a primeira geração do macOS.

Esses computadores pessoais precisavam acessar bancos de dados, arquivos e programas corporativos centralizados em um computador servidor, inicialmente em redes locais (LAN) e posteriormente na internet, a grande rede mundial de computadores, utilizando um modelo de computação cliente-servidor.

Na década de 1990, há o surgimento da World Wide Web, ou simplesmente web, incorporando à internet páginas com hipertexto escritas em linguagem HTML, constituindo os websites, o protocolo HTTP e os programas navegadores para acessar esses novos recursos. Os computadores pessoais se tornam muito mais disseminados ao longo da década, empregando SOs com interfaces gráficas, e, na passagem para os anos 2000, a evolução e o barateamento das redes de comunicação popularizam tanto o acesso à internet quanto o uso da web. As aplicações clientes vão se tornando os navegadores e as aplicações servidores passam a estar disponíveis continuamente através da internet. Para atender à demanda crescente, a infraestrutura de TI corporativa começa a migrar para locais planejados para esse tipo de operação desde a construção física, o fornecimento de energia e a refrigeração até a conectividade rápida e ininterrupta com a internet e a possibilidade de comportar alta capacidade de processamento e armazenamento de dados, além do emprego de tecnologias de virtualização. Esses locais, que já foram chamados de Central de Processamento de Dados (CPD), são mais conhecidos atualmente apenas como centro de dados ou, pelo termo em inglês “data center”.

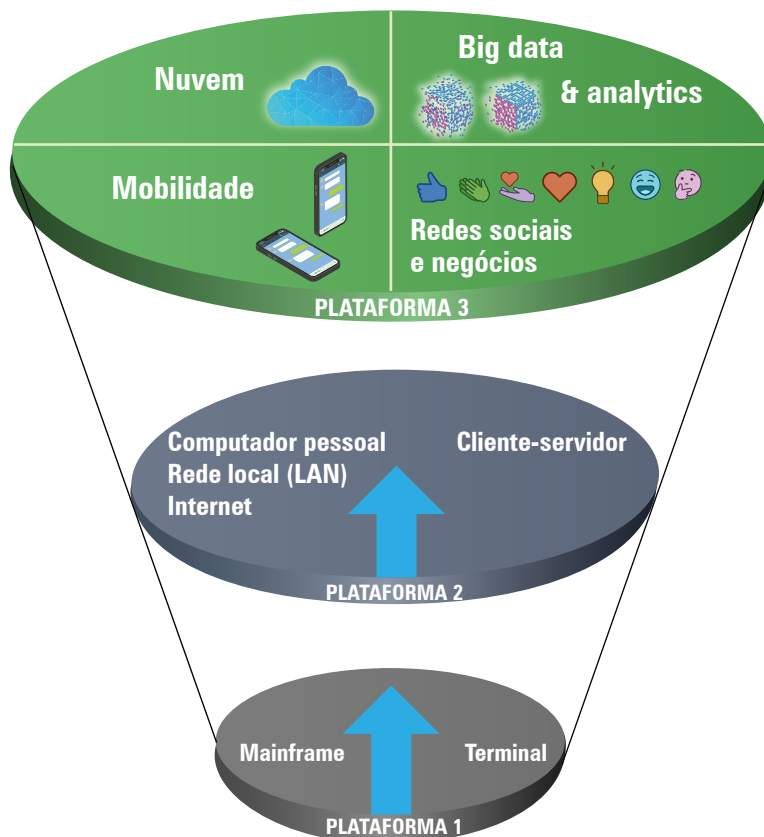
Nessa década, surge também o movimento de código aberto, open source, ou software livre, que preconiza que as pessoas devem ter acesso ao código fonte dos programas para poder estudar, auditar e mesmo modificar. Sem entrar nas nuances que possam diferir esses e alguns outros termos, é importante ressaltar duas coisas. Primeiro que boa parte desse movimento pressupõe que o software deva ser gratuito, com o sustento financeiro sendo obtido por meio de serviços como suporte, e treinamento. O segundo ponto é que isso acelerou grandemente o avanço tecnológico, pois as principais ferramentas de software associadas à

web se encaixam nesse modelo, além de linguagens de programação e ferramentas de desenvolvimento, aplicações de big data & analytics e assim por diante. Essa lista deve incluir o Linux, um SO desenvolvido na década de 1990 pelo finlandês Linus Torvalds. O Linux é um SO de código aberto projetado para ser compatível com o Unix e que, ao longo da década de 2010, passaria a ser o SO mais utilizado no mundo.

Essa fase é referida como “segunda plataforma computacional” e obviamente não acabou. Até mesmo os mainframes, da primeira plataforma, ainda são utilizados em setores como bancário, financeiro e governamental para uma série de aplicações. Mas essa demanda por mais recursos de TI disponíveis de forma contínua e ubíqua traz problemas intrínsecos como tempo de implementação, escala, dificuldade de gerenciamento e operação e custos. Para tentar amenizar esses problemas surgiu um novo modelo de TI como serviço que é a computação em nuvem (cloud computing). O primeiro provedor de nuvem foi a Amazon Web Services (AWS), lançado em 2006.

A computação em nuvem compõe a “terceira plataforma computacional” e é considerada um de seus quatro pilares, juntamente com computação móvel, big data & analytics e redes sociais e negócios. A computação móvel envolve não apenas smartphones, mas também a possibilidade de se “computar” em movimento mantendo-se uma conexão por meio de redes sem fio como 4G, 5G e mesmo Wi-Fi. Big data & analytics começou com a necessidade de se organizar o armazenamento e a recuperação de informações em bases de dados muito grandes, como os buscadores de internet, e hoje são essenciais para todas as organizações. Redes sociais e negócios vão além de plataformas como Facebook e LinkedIn, mas passam também pela incorporação de elementos sociais em todos os outros negócios no mundo digital. A figura 2 representa essas três plataformas computacionais.

Figura 2 – As três plataformas computacionais



Fonte: Adaptado de IDC (s. d.).

É bastante fácil pensar em sistemas de terceira plataforma de computação que se utilizam desses quatro pilares. Aplicações de navegação como Waze e Google Maps são executados em servidores em nuvem, para que possam ser acessados de forma contínua e móvel por smartphones e equipamento multimídia conectados a redes 4G e 5G em veículos, fornecendo as melhores rotas com base na análise em tempo real de um grande volume de dados e utilizando informações e reações em tempo real de sua rede social de usuários. Plataformas como Blackboard, Moodle e Canvas, utilizados por universidades, escolas e empresas para

seus cursos à distância e suporte a cursos presenciais, também são exemplos disso, assim como a maioria das aplicações disponíveis em smartphones, smart TVs e outros dispositivos inteligentes. A internet das coisas, ou IoT na sigla correspondente em inglês, também está inserida nesse contexto e, mais recentemente, o “boom” das plataformas de inteligência artificial (IA) generativa.

Os SOs da segunda plataforma continuam sendo usados na terceira plataforma com o Linux dominando as áreas de computação de alto desempenho, computação em nuvem, big data & analytics e mobilidade através do Android, sistemas de smart TV e muitos sistemas de IoT. O iOS e o macOS são utilizados pela Apple e são sistemas Unix, enquanto o Windows, da Microsoft, é o sistema dominante em computadores pessoais e um forte competidor em equipamentos embarcados e servidores corporativos. Mas há uma imensa variedade de outros SOs nas áreas de IoT, robótica, automação, transporte, telecomunicações. Muitos deles são baseados no Unix.



PARA SABER MAIS

Uma versão mais detalhada da história dos sistemas operacionais pode ser encontrada nos livros dos professores Tanenbaum e Bos (2024), Machado e Maia (2013) e de Deitel, Deitel e Choffnes (2005). Uma visão bastante abrangente de todos os aspectos da história da computação pode ser lida no livro do professor Wazlawick (2016). Além disso, o jornalista Walter Isaacson (2014) apresenta a revolução digital em uma linguagem mais popular.

3 Tipos de sistema operacional

Há muitas formas de se classificar os sistemas operacionais. Tanenbaum e Bos (2024) classificam os SOs conforme o computador a que

se destinam: computadores de grande porte, servidores, computadores pessoais, smartphones e computadores portáteis, internet das coisas e embarcados, tempo real e cartões inteligentes (smartcards). Uma divisão similar é feita por Maziero (2019): *batch* (de lote), de rede, distribuído, multiusuário, servidor, desktop, móvel, embarcado e tempo real. Já Machado e Maia (2013) classificam os SOs conforme alguns parâmetros técnicos como: mono ou multiusuário, mono ou multiprogramável e o tipo de suporte a múltiplos processadores.

Essas diferentes maneiras de se classificar um SO possuem pontos positivos e negativos. Um aspecto que torna qualquer tipo de classificação discutível é a realidade prática, uma vez que o Linux se aplica a praticamente todas as categorias nessas classificações. Mesmo o Windows possui versões que se encaixam em quase todas as categorias. A imensa maioria do mercado hoje se concentra na “família” Unix, incluindo aí o Linux, Android, macOS e iOS e nas diferentes versões do Windows. É claro que há uma grande diversidade de outros SO, mas acabam sendo ou para aplicações mais específicas, ou para computadores com baixo poder computacional, ou devido a restrições mercadológicas como a proibição do uso do Android imposta à fabricante chinesa Huawei, que acabou desenvolvendo seu próprio sistema operacional.

Assim, em termos mais práticos, pode-se propor uma classificação simplificada dos SOs em sistemas servidores e sistemas desktop, sendo que nos sistemas desktop ainda é possível considerar um grupo específico de sistemas para computadores móveis. Embora continue apresentando a mesma restrição prática de Linux, ou Windows se encaixarem em todos os grupos, é possível fazer algumas distinções claras que afetam o dia a dia do desenvolvedor, do administrador e mesmo do usuário:

- **SO servidor:** executado em uma máquina servidora, aquela que está ligada “24 x 7”, apta a receber requisições que serão atendidas pelos programas servidores, como o servidor web e servidor de banco de dados. Geralmente essas máquinas estão em data

centers e não possuem um operador que as utilize diretamente através de um teclado, mouse e monitor, sendo acessadas de forma remota. Com a necessidade de maior segurança cibernética e menor consumo de recursos computacionais e de energia, é prevalente nessa categoria que não se utilize interfaces gráficas de usuário, mas apenas interfaces textuais. Do ponto de vista de aplicações é o SO do back-end. Dispositivos de IoT e computadores embarcados também utilizam esse tipo de sistema.

- **SO desktop (e SO para dispositivos móveis):** possuem interface gráfica de usuário para serem mais amigáveis e são operados diretamente por um usuário final através de teclados, mouse, canetas, telas de toque e monitores. Demandam mais recursos computacionais para executar a interface gráfica e frequentemente têm que lidar com um volume de periféricos muito maior e nem sempre da melhor adequação às normas técnicas. O termo “desktop” faz relação à mesa de trabalho. Máquinas de escrever, calculadoras, calendário, agendas, aparelhos de fax, planilhas manuais, entre outras ferramentas que ocupavam as mesas de trabalho, foram sendo incorporadas aos computadores por meio de aplicações como o pacote de escritórios. É nesse sentido que sistemas móveis como Android (Linux) e iOS (Unix) também podem ser considerados sistemas desktop. Do ponto de vista de aplicações é o SO do front-end.

4 Preparando um sistema para a prática ao longo do curso

Conforme mencionado, o Linux é o SO mais usado no mundo atualmente e segue os padrões dos sistemas Unix. Para se acompanhar a parte prática deste curso, além da possibilidade de se usar um computador com o Linux, ou outro Unix, efetivamente instalado, é possível utilizar uma série de ferramentas: o GitHub Codespaces para aqueles que

já possuem uma conta no GitHub, um provedor de nuvem como a Microsoft Azure e Amazon AWS, uma máquina virtual (VirtualBox, Hyper V, etc.) com o Linux instalado, o Windows Subsystem for Linux (WSL) no Windows, etc. Quanto à distribuição, usaremos a Ubuntu, que é a mais usada no mundo atualmente.

Considerações finais

Neste capítulo, estudamos o que é um sistema operacional, como eles evoluíram e se inserem no atual cenário tecnológico e os tipos de sistema operacional existentes. Quais os sistemas operacionais que você utiliza no dia a dia? Já teve a curiosidade de visitar o site oficial dele, ou de ler uma parte da documentação que o acompanha?

Por fim, foram abordadas as possibilidades para praticar com o sistema operacional Linux. Esse ponto deixa uma “lição de casa” importante para a sequência dos estudos. Mãos à obra!

Referências

DEITEL, H. M.; DEITEL, P. J.; CHOFFNES, D. R. **Sistemas operacionais**. 3. ed. São Paulo: Pearson, 2005.

INTERNATIONAL DATA CORPORATION – IDC. [S. d.]. Disponível em: <https://www.idc.com/>. Acesso em: 2 fev. 2025.

ISAACSON, W. **Os inovadores**: uma biografia da revolução digital. Tradução Luciano Vieira Machado, Pedro Maia Soares e Berilo Vargas. São Paulo: Companhia das Letras, 2014.

MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. 5. ed. Rio de Janeiro: LTC, 2013.

MAZIERO, C. A. **Sistemas operacionais**: conceitos e mecanismos [recurso eletrônico]. Curitiba: DINF; UFPR, 2019. Disponível em: <http://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start>. Acesso em: 29 jan. 2025.

TANENBAUM, A. S.; BOS, H. **Sistemas operacionais modernos**. 5. ed. Porto Alegre: Bookman, 2024.

WAZLAWICK, R. S. **História da computação**. São Paulo: GEN LTC, 2016.

Arquitetura dos sistemas operacionais

Quando pensamos em um sistema operacional, como Android, Windows, ou macOS, acabamos por considerar um “produto” de software bastante amplo, que, quando instalado no computador nos permite realizar diversas tarefas, indo muito além da ideia de um simples programa de controle. O que acontece é que esses “produtos” SO já vem acompanhados de navegadores web, leitores de PDF, visualizadores e editores de imagem e vídeo, gravadores de áudio, programas de câmera, etc. Ou seja, vêm com diversas aplicações para o usuário final. Mas elas são “acessórios”, ou complementos, e não a parte principal que vai controlar o hardware e as aplicações, o cerne, o núcleo! Ou, em inglês, o kernel. Como a evolução dos sistemas operacionais utilizados em computadores pessoais e servidores, o kernel precisou realizar cada vez mais tarefas e foi se tornando maior e mais complexo. Com isso, a maioria dos SOs atuais tem parte de suas funções realizadas no espaço de usuário, ou userspace em inglês.

Voltando a pensar no “produto” SO, além do kernel, do espaço de usuário e de algumas aplicações para o usuário final, há sempre uma série de utilitários. Uma forma de definir os utilitários são como programas “meio”, e não “fim”, ou se preferir são como ferramentas. Exemplos de utilitários são o gerenciador de arquivos, gerenciador de tarefas, programa de configuração, loja de aplicativos, editor de código, compilador, depurador, etc. Todo sistema operacional precisa também de uma forma de interação com o usuário, seja através de comandos digitados, seja de forma gráfica e amigável, a chamada interface.

A seguir, vamos explorar a arquitetura dos sistemas operacionais atuais e de que forma o sistema operacional é inicializado.

1 O núcleo (kernel)

O kernel do sistema operacional é o responsável por gerenciar os processos, isto é, os programas em execução, e por todas as tarefas do gerenciamento do hardware. Por exemplo, o tratamento de interrupções e exceções, o tratamento dos sinais, o acesso ao sistema de arquivos, o acesso aos dispositivos periféricos como placa de rede, áudio, teclado, mouse, monitor. Também é responsável pelo gerenciamento da memória principal e dos sistemas de arquivos, a memória secundária. Cada uma dessas funções é realizada por uma rotina, formando o kernel em si. Ele é o programa principal do sistema operacional e normalmente é um único código executável.

Para que o núcleo do SO exerça suas funções, ele precisa ser executado em modo privilegiado, isto é, podendo executar determinadas instruções de máquina que outros programas não podem. Esse modo privilegiado de funcionamento da CPU é chamado justamente de modo kernel, enquanto outros programas funcionam em modo usuário. Em modo kernel, é possível executar as instruções, que inibem interrupções, protegem regiões de memória, alteram a frequência de *clock* da CPU, etc.

Para que o kernel do SO garanta que nenhum outro programa possa funcionar no mesmo modo, ele precisa ser o primeiro programa a ser executado. Mas ele é um programa! E precisa ser executado... Abordaremos esse ponto logo mais, mas primeiro vamos entender de que forma é possível interagir com o kernel.

Para solicitar qualquer serviço ao SO, ou seja, fazer solicitações ao kernel, é necessário usar um mecanismo de requisição conhecido como system call, ou “chamada de sistema” em português. Por exemplo, para as funções de gerenciamento dos processos, há chamadas de sistema específicas para criar um processo, eliminar um processo, criar um processo filho, alterar características do processo, retornar o status de um processo após sua conclusão e assim por diante (Machado; Maia, 2013; Tanenbaum; Bos, 2024).

Nos sistemas Unix e derivados há normas técnicas para as system calls para viabilizar a portabilidade de aplicações. Trata-se do Portable Operating System Interface (POSIX), definido pelo Instituto de Engenheiros Eletricistas e Eletrônicos (IEEE) e ratificado pela Organização Internacional de Normalização (ISO) (Tanenbaum; Bos, 2024).

As chamadas de sistema são geradas o tempo todo, desde o próprio comando para executar o programa e durante toda a sua execução. Quando o kernel recebe a chamada de sistema do processo ele (re) assume o controle do computador, passa o processador para o modo privilegiado (modo kernel), atende a solicitação, retorna o processador ao modo usuário e faz com o processo que havia solicitado a system call volte a ser executado. Quando um desenvolvedor está escrevendo um programa são utilizados métodos e funções para, por exemplo, acessar o hardware, ler um clique de mouse, executar outro programa, abrir ou salvar um arquivo. Esses métodos e funções são convertidos nas chamadas de sistema no momento da compilação, ou interpretação. Por fim, quando é o kernel que precisa se comunicar, ele envia um sinal para o processo.

2 O espaço do usuário

Conforme os núcleos dos SOs evoluíram, eles foram ficando mais complexos e maiores. E muitas das tarefas de gerenciamento de um SO puderam ser executadas em modo usuário e fora do código executável do kernel. Tarefas como gerenciamento de redes e impressão são alguns desses exemplos. Essa abordagem constitui uma arquitetura em camadas simplificada e é a arquitetura mais usada nos SOs disponíveis hoje no mercado, incluindo o Windows e a maior parte dos derivados do Unix (Machado; Maia, 2013). Essas duas camadas são conhecidas por espaço de usuário (userspace) e kernel.

O termo “espaço de usuário” pode passar a impressão de corresponder aos programas do usuário final, mas na verdade refere-se apenas às partes do SO que não precisam ser executadas em modo privilegiado. As aplicações finais são executadas sobre o SO, compreendendo as duas camadas. Uma discussão sobre outras arquiteturas de sistemas operacionais pode ser encontrada em Tanenbaum e Bos (2024).

3 A interface com o usuário

A palavra “interface” é literalmente algo que está entre duas faces. O próprio SO é uma interface com o hardware subjacente. A interface com o usuário final pode ser gráfica ou textual. As interfaces gráficas são conhecidas pela sigla em inglês GUI, de graphical user interface. O Windows tem sua interface gráfica padrão, assim como o macOS.

Já nos smartphones, sistemas Android de diferentes fabricantes costumam possuir GUIs diferentes, ou ao menos modificações substanciais da GUI padrão, como a One UI, da Samsung. Sistemas Unix e Linux possuem GUIs populares, como Gnome e KDE. Nesses últimos exemplos, a mesma GUI pode ser usada por diferentes SOs e elas também são chamadas de Ambiente Desktop, por incluírem uma série de

ferramentas de desenvolvimento, bibliotecas de suporte para aplicações gráficas (GTK e Qt, por exemplo), aplicações, etc. As interfaces gráficas costumam ser mais amigáveis ao usuário final, porém consomem muito mais recursos do sistema e acabam resultando em um sistema mais complexo e suscetível a problemas.

A interface mais básica, desde os tempos dos computadores da primeira plataforma computacional, é aquela baseada em comandos, ou CLI, de command-line interface. Esse tipo de interface utiliza um programa interpretador de comandos e suporta a automação de tarefa por meio de programas chamados scripts. O interpretador de comandos é conhecido também como shell. Pense na ideia de uma casca, ou concha, que está entre o usuário e o núcleo. O termo “shell” também pode ser usado para se referir às GUIs, mas é menos comum. As CLIs também podem ser usadas dentro das GUIs através de uma aplicação terminal. Já em sistemas servidor, geralmente tem-se uma interface apenas com linhas de comando.

É possível haver mais de uma shell em um SO. O Windows possui duas shells padrão, o prompt de comando, herança dos tempos do DOS, e a PowerShell, mais recente. Nos sistemas Unix e Linux, a shell mais popular é o Bash. Ela é a mais usada no meio corporativo e a que será encontrada mais amplamente nas documentações técnicas, além de ser totalmente compatível com o padrão POSIX. Mas existem outras shells, como ZSH e Fish, e mais de uma pode estar disponível em um sistema.

É importante notar que a CLI é uma interface para o SO, isto é, o núcleo e o espaço de usuário. Dessa forma, é possível instalar o Bash no Windows, ou o PowerShell, no Linux. Ao digitar-se um comando e pressionar a tecla Enter, ou clicar em um ícone para executar um programa, o que aconteceu foi a geração das chamadas de sistema apropriadas para o núcleo do SO.



NA PRÁTICA

Criado pelo cientista de computação finlandês Linus Torvalds no começo da década de 1990, o Linux é, na verdade, o kernel do sistema (Kernel, [s. d.]). O kernel Linux é compatível com o padrão POSIX, por isso pode-se dizer que o Linux é um SO compatível com o Unix. Quando a primeira versão foi lançada, eram necessários utilitários como interpretador de comandos Bash, compilador GCC e editor de textos. Esses utilitários básicos para o sistema operacional foram obtidos a partir do projeto GNU (GNU, [s. d.]), do movimento open source, por isso muitas vezes é referido como um sistema GNU/Linux. O kernel também utiliza uma licença de código aberto. Conforme foi se desenvolvendo e ganhando popularidade, esse sistema passou a incorporar mais utilitários – ferramentas para gerenciamento de pacotes (aplicações e atualização do sistema), gerenciamento e compactação de arquivos, programação, interfaces gráficas e programas de configuração –, além de aplicações para o usuário final, como navegadores, pacotes de escritório, entre outros. Dessa maneira, diversas empresas ou comunidades de usuários começaram a fazer suas distribuições Linux, ou seja, versões completas do “produto” sistema operacional com base no kernel Linux.

Considerando os sistemas do tipo servidor, as principais diferenças entre as distribuições abrangem o gerenciamento de pacotes, configurações de rede e segurança, entre outras ferramentas de configuração. Em um sistema desktop, soma-se a essas diferenças a interface gráfica utilizada (Gnome, KDE, Mate, LXQt, por exemplo), o conjunto de aplicações e utilitários entregues ao usuário final, além de ajustes estéticos, como fundos de tela, temas e fontes. A maioria das distribuições Linux atuais utiliza o systemd como ferramenta para inicialização e gerenciamento do espaço de usuário.

As principais distribuições Linux utilizadas atualmente são a Ubuntu (<https://ubuntu.com>), que, por sua vez, é uma derivada do Debian e, com diversas outras, compartilham as ferramentas de gerenciamento de pacotes apt e o formato de pacotes deb, além de diversas outras ferramentas de configuração em comum.

A RedHat, adquirida pela IBM, ao lado de softwares como Oracle Linux, Amazon Linux e Fedora, forma a outra grande família de distribuições, compartilhando a ferramenta de gerenciamento de pacotes dnf e o formato de pacotes rpm, além de compartilharem outro conjunto de ferramentas de configuração em comum. Essas distribuições mencionadas

possuem versões de servidor e desktop e são de amplo uso no meio corporativo.

Linus Torvald ainda é o líder do desenvolvimento do kernel Linux, trabalhando na Linux Foundation, que também é responsável por centenas de outros importantes projetos de software livre.

Além das distribuições Linux, os sistemas operacionais Android, Chrome OS e de smart TVs também utilizam o kernel Linux, fazendo com que, desde a década de 2010, o Linux passasse a ser o SO mais usado do mundo.

4 A inicialização do sistema, ou como se levantar segurando as alças de suas botas!

Se GUIs e CLIs são usadas para acionar a criação de um processo e, a partir disso, a execução dos programas, como um SO em si pode ser executado? Essa resposta envolve alguns recursos de hardware e software. Ao energizar (ligar) um computador, a CPU passa a executar o programa carregado na posição inicial de memória, então é preciso garantir que haja um programa adequado já carregado na memória. Mas a memória do computador é volátil, isto é, perde os dados sem energia. Então é preciso haver um pouco de memória não volátil, normalmente uma memória do tipo Flash com um software básico para ser executado inicialmente. Esse programa é chamado firmware.

O Basic Input Output System (BIOS) foi uma versão muito popular usada nas primeiras décadas dos computadores IBM-PC e compatíveis, sendo substituída pelo padrão EFI, especialmente o Unified Extensible Firmware Interface (UEFI), que é usado na maior parte dos tipos de computador atuais. Assim como o ícone “salvar” continua sendo um velho disquete, o nome “BIOS” continua sendo mencionado, mesmo após o programa em si não estar mais em uso (Linux, [s. d.]; Tanenbaum; Bos, 2024).

O firmware faz uma breve verificação do hardware e busca por um dispositivo de armazenamento que contenha uma partição inicializável com outro inicializador de sistemas operacionais, o bootloader. O bootloader finalmente é responsável por carregar o kernel do SO na memória e, a partir daí, o kernel do SO assume o controle da máquina, iniciando o espaço de usuário e todas as demais aplicações (Linux, [s. d.]; Tanenbaum; Bos, 2024).

Frequentemente, o bootloader possui um menu com opções adicionais para iniciar o SO em um modo de segurança ou manutenção, sem a interface gráfica, por exemplo. Os bootloaders podem iniciar mais de um sistema operacional. No Windows, o bootloader pode ser usado para carregar um Linux e vice-versa. Os bootloaders mais populares no Linux são o GRUB e o systemd-boot. O padrão UEFI pode ser usado para carregar um kernel de SO diretamente, embora seja menos comum.

Talvez você esteja se perguntando o que as botas (boot) têm a ver com a história? Bom, o termo “boot”, neste contexto, vem literalmente de “botas”. Há uma expressão na língua inglesa que é *“pull oneself up by one’s (own) bootstraps”* (Spears, 2002). Ela dá uma ideia de se obter sucesso, especialmente em uma empreitada difícil, pelos próprios esforços ou, de uma forma mais literal, se levantar segurando as alças das próprias botas... Parece apropriado, não?

5 Prática: comandos básicos do Linux e do Unix

O Bash, interpretador de comandos, exibe informações antes do símbolo “\$”, chamado prompt, seguido de um cursor piscando para indicar que o sistema está pronto para receber um comando do usuário. O aspecto deve ser similar ao da figura 1. Temos o nome do usuário, o “@”, que significa a preposição “em” (como usada nos e-mails, por exemplo), o nome da máquina, o “.”, seguido do caminho do diretório (pasta) atual, e o “\$”, que

indica que somos um usuário convencional. Caso o caractere final seja um “#”, indica que somos um usuário administrador, que, em sistemas Unix, é chamado root.

Figura 1 – Composição típica de um prompt Bash



```
usuario@nome-da-maquina:/caminho/diretorio/atuall$
```

Em alguns sistemas, pode não haver nenhuma informação, apenas o “\$” ou “#”. Para obtê-las, o usuário pode digitar os comandos `whoami`, `hostname` e `pwd`, pressionando a tecla Enter na sequência.

Para navegar entre os diretórios, pode-se usar o comando `cd`. Experimente as combinações a seguir sem o uso das aspas e pressione a tecla Enter. Atente-se também aos espaços. À medida que executar os comandos, observe as informações no prompt: “`cd ..`”, “`pwd`”, “`cd`”, “`pwd`”, “`cd /`”, “`cd ~`”. Note que o uso de “`..`” leva para o diretório superior, e “`~`” não será lido como “til”, mas como “home”, o diretório de trabalho do usuário. Execute “`cd /`” e agora o comando “`ls`”. Ele lista o conteúdo do diretório. O diretório “`/`”, ou diretório raiz, contém todos os outros diretórios. Cada um deles tem uma finalidade específica. Você pode pesquisar por “Linux FHS –Filesystem Hierarchy Standard” para saber mais sobre eles.

Para obter ajuda sobre um comando, você pode utilizar “comando `--help`” ou “`man comando`”. Por exemplo: “`ls --help`” ou “`man ls`”. Para sair do texto de ajuda no comando “`man`”, use a tecla “`q`”.

Agora, consulte a ajuda e teste os comandos “`uname`”, “`uname -a`” e “`hostnamectl`”. Eles exibem informações relacionadas a alguns itens estudados anteriormente.

Considerações finais

Neste capítulo, entendemos que a interação do usuário com o sistema ocorre por meio de uma GUI ou CLI. As interações do usuário e a execução dos programas vai gerar uma série de chamadas de sistema (system calls), pois é o núcleo (kernel) do SO que gerencia tanto as aplicações quanto o hardware, ou seja, faz as coisas acontecerem. O núcleo do SO é executado em modo privilegiado e conta com diversos serviços no espaço de usuário (userspace), em modo não privilegiado. Por fim, o SO conta com o suporte de um firmware e mais um bootloader, para ser encontrado, carregado na memória e executado, assumindo o controle da máquina.

Referências

- GNU. [S. d.]. Disponível em: <https://www.gnu.org>. Acesso em: 13 fev. 2025.
- KERNEL. [S. d.]. Disponível em: <https://www.kernel.org>. Acesso em: 13 fev. 2025.
- LINUX. **bootup(7) – Linux manual page**. [S. d.]. Disponível em: <https://man7.org/linux/man-pages/man7/bootup.7.html>. Acesso em: 13 fev. 2025.
- LINUX FOUNDATION. [S. d.]. Disponível em: <https://www.linuxfoundation.org/>. Acesso em: 13 fev. 2025.
- MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. 5. ed. Rio de Janeiro: LTC, 2013.
- SPEARS, R. A. Pull oneself up by one's (own) bootstraps. *In: McGraw-Hill Dictionary of American Idioms and Phrasal Verbs*. Nova York: McGraw-Hill, 2002.
- TANENBAUM, A. S.; BOS, H. **Sistemas operacionais modernos**. 5. ed. Porto Alegre: Bookman, 2024.

Arquitetura de computadores: uma visão do suporte ao sistema operacional

Os computadores, dos mais simples aos mais avançados, possuem: uma estrutura básica comum; um ou mais processadores, ou unidade central de processamento (CPU – central processing unit); dispositivos de entrada e de saída, normalmente abreviados por “E/S”, ou “I/O”, de input/output. Além disso, essas máquinas possuem as memórias, ou sistema de armazenamento, em que temos ao menos uma memória de trabalho para armazenar os programas em execução e seus respectivos dados, e uma memória maior e permanente, que armazena todos os programas e dados. Usamos os termos técnicos “memória principal” e “memória secundária” para nos referirmos a elas.

Nesse sentido, a memória principal utiliza semicondutores do tipo RAM dinâmica (DRAM). RAM significa memória de acesso aleatório (ou random access memory) e corresponde a um tipo de circuito de integrado. Essas memórias são voláteis, isto é, perdem os dados quando desligadas. Apesar de continuarmos a usar o termo “disco”, as memórias secundárias deixaram de ser redondas, muito embora você ainda possa encontrar HDs, CDs e DVDs. Os sistemas mais modernos costumam contar apenas com dispositivos baseados em semicondutores do tipo Flash, outro modelo de circuito integrado de memória que não é volátil. As memórias Flash estão presentes em SSDs, cartões de memória, pen drives, entre outros.

Nos computadores que executam sistemas operacionais e possuem capacidade de multitarefa, isto é, ter mais de um programa em execução, também há as memórias cache, que são uma pequena porção de memória muito rápida, posicionada entre a CPU e a memória principal para melhorar o desempenho do sistema. A memória cache é totalmente controlada por hardware, sendo fabricada com circuitos integrados do tipo RAM, mas, nesse caso, RAM estática (SRAM), o que permite que sejam feitas com as CPUs no mesmo circuito integrado microprocessador.

Quando os microprocessadores foram criados, o termo era usado para uma CPU em um único circuito integrado (chip). Atualmente, os microprocessadores podem contar com muitas CPUs, memória cache, unidade de processamento gráfico (GPU), unidade de processamento neural (NPU), entre outros circuitos. Nesse contexto, é relativamente comum usar as abreviações SoC, de “system on chip”, e SiP, de “system in package”.

O sistema de E/S engloba sempre um módulo de E/S e um ou mais dispositivos de E/S, também chamados periféricos. O módulo de E/S equivale a controlador, ou placa controladora. Exemplos desses pares são a placa de som e os alto-falantes, caixas de som e headsets; placa de vídeo e monitores e projetores; controladores de disco, SSDs, entre outros.

Para constituir um computador, é necessário interligar todos esses componentes mencionados de maneira adequada. É para isso que existem os barramentos, que proveem um meio de comunicação padronizado e frequentemente compartilhável para diversos tipos de componente em um computador. Os padrões incluem características lógicas, eletroeletrônicas, como especificações de sinais, níveis de tensão e frequências, e mecânicas, como conectores.

Por isso, reconhecemos facilmente um conector USB e o associamos à tensão padrão de 5 V, utilizada para carregar nossos “gadgets”. A propósito, USB significa barramento serial universal (universal serial bus) e é o barramento responsável por conectar principalmente os componentes do computador que estão em contato com um usuário final, tais como teclado, mouse, pen drive, câmera, etc.



IMPORTANTE

Há duas técnicas muito usadas para a melhoria de desempenho nos computadores e que são empregadas de diversas formas. A primeira é o paralelismo. A ideia de fazer mais de uma coisa ao mesmo tempo. Usa-se nos computadores com múltiplas CPUs, nos links de dados paralelos em um barramento ou rede de computadores, no sistema de armazenamento e na programação.

A outra técnica consiste em se inserir um intermediário rápido entre algo que tem um desempenho superior (“rápido”) e outro que tem um desempenho inferior (“lento”). Normalmente esse intermediário rápido é relativamente pequeno para poder manter um bom desempenho. É esse o princípio das memórias cache, do buffer da placa de rede, do mecanismo de buffering de software – para banco de dados, por exemplo – e do mecanismo de spooling de impressão, em que a memória secundária é esse intermediário rápido entre a CPU e a impressora.

1 Atuação do sistema operacional sobre o hardware

Nos computadores de propósito geral, os sistemas operacionais atuam sobre o hardware por meio de quatro tarefas de gerenciamento principais:

1. **Gerenciamento do processador, em especial o escalonamento**

De um ponto de vista prático, executar programas é a principal tarefa de um computador. Normalmente, há diversos programas em execução, mais do que a quantidade de CPUs disponíveis. Então é preciso definir quais programas serão executados em quais CPUs a cada momento, utilizando critérios adequados implementados por meio de um algoritmo de escalonamento. Para gerenciar as CPUs, o SO modifica o conteúdo do registrador Program Counter da CPU e se vale de recursos de pilha de memória e bancos de registradores de controle.

2. **Gerenciamento de memória, em especial a memória virtual**

Se há vários programas em execução, é necessário que eles estejam carregados na memória principal. Então, é preciso dividir, ou compartilhar, a memória entre vários programas e os dados e arquivos sendo manipulados por esses programas. Também é necessário aceitar novos programas em execução (processos) e arquivos. Não é uma tarefa tão simples, pois os programas e arquivos podem ser muito grandes para os espaços disponíveis, que, eventualmente, podem ser muito pequenos. Além disso, pode até acontecer de simplesmente não haver espaço disponível na memória principal. Nesse último caso, é necessário “pegar emprestado” espaço na memória secundária, mecanismo conhecido como “swap”. Para tornar o gerenciamento da memória mais eficiente, os sistemas empregam uma técnica chamada de memória virtual. Para

implementar a memória virtual, os sistemas operacionais contam com o apoio dos circuitos controladores de memória no hardware.

3. **Gerenciamento de sistema de arquivos**

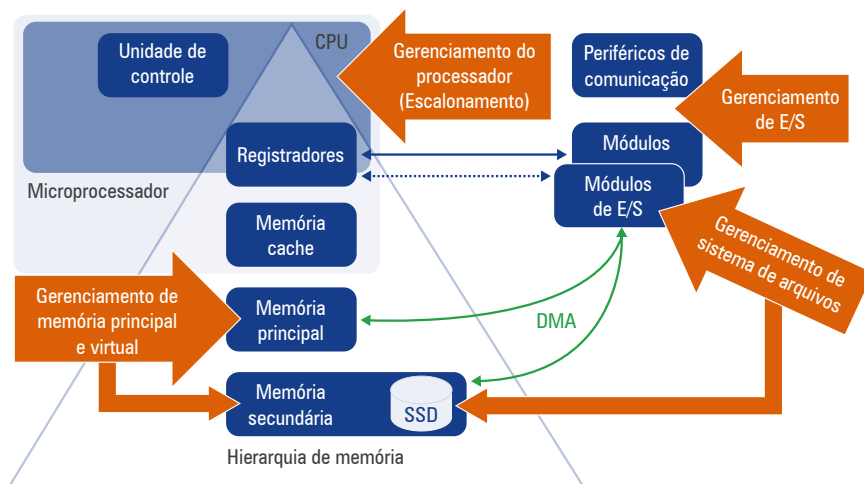
O sistema operacional é um software e, portanto, seu código executável precisa ser armazenado na memória secundária na forma de arquivos. Da mesma forma, todas as aplicações e arquivos que ele gerencia. Um computador pode contar com diversos dispositivos de memória secundária, conforme descrito na introdução deste capítulo. Com todos eles, tanto um usuário final quanto um desenvolvedor de software trabalham com a mesma analogia de diretórios e arquivos em uma organização lógica conhecida como sistema de arquivos. Há diversos exemplos de sistemas de arquivos, como FAT32, NTFS e EXT. A forma de trabalho é a mesma, independentemente do tipo de dispositivo de armazenamento e padrão de barramento utilizado. Essa é uma abstração, ou virtualização, fundamental para se facilitar o uso dos computadores desde a inicialização do sistema.

4. **Gerenciamento de E/S**

Para atuar sobre um periférico de saída, o SO precisa enviar os comandos adequados para o módulo de E/S correspondente. Para ler os dados de um periférico, o SO precisa obtê-los do módulo de E/S responsável pelo periférico, utilizando-se dos comandos adequados. São muitos módulos de E/S e muitos periféricos diferentes, por isso o SO precisa de verdadeiros manuais de como ler e escrever em cada um deles, o que é representado pelos drivers de dispositivo (device drivers).

A seguir, a figura 1 traz uma representação gráfica de como essas tarefas de gerenciamento estão relacionadas com a estrutura de um computador.

Figura 1 – Atuação do sistema operacional sobre o hardware



Fonte: Valiante Filho (2020).

2 Concorrência, interrupções e exceções

Um conceito bastante importante relacionado aos sistemas operacionais é o de “concorrência”. Caso consulte essa palavra no dicionário, notará significados que são verdadeiros antônimos. “Disputa”, “competição”, “simultaneidade” e “concordância” estão entre as acepções do termo. Nos sistemas operacionais, todas essas acepções do termo acabam fazendo sentido. A concorrência é efetivamente a possibilidade de compartilhar os diversos recursos de um computador, causando uma disputa por esses recursos. Mas, ao mesmo tempo, é necessário que essa competição seja extremamente leal, pois poderíamos dizer que qualquer desrespeito às regras prejudicará todos os competidores nesse caso.

Silberschatz (2015, p. 93) define que “um sistema concorrente dá suporte a mais de uma tarefa, permitindo que todas elas progridam. Assim, é possível haver concorrência sem paralelismo”. Isso implica revezar o uso dos recursos de hardware do computador.

Há diversos recursos para a implementação da concorrência nos sistemas computacionais (Machado; Maia, 2013). Exemplo disso são técnicas modernas para as operações de E/S, como direct memory access (DMA), direct cache access (DCA), buffering, spooling, reentrância e, principalmente, as interrupções e exceções.

É possível fazer um breve exercício de imaginação para entender o conceito de interrupção. Você está estudando este material, processando cuidadosamente as informações. Agora imagine que, inadvertidamente, você se esqueceu de desligar seu smartphone ou colocá-lo no modo “não incomodar”. Então, ele tocou interrompendo seu estudo e, com isso, você deve tomar o cuidado de marcar em que ponto estava e atender à interrupção, digo, ligação. Depois de resolvido o assunto, você pode retomar o estudo e continuar de onde havia parado.

Esse mecanismo existe nos computadores. A interrupção (INT) permite fazer com que o computador interrompa o programa em execução e vá executar outro programa. É importante frisar que o computador não para de executar instruções em nenhum momento. As interrupções apenas forçam uma troca do programa em execução. Nesse caso, o programa que passa a ser executado é a rotina de tratamento de interrupções (RTI), que é parte do núcleo do sistema operacional. Depois que o sistema operacional trata a interrupção, o programa anterior pode ser retomado normalmente, ou mesmo outro programa, para que depois o programa interrompido possa ser retomado. Em inglês, a RTI é indicada pela sigla ISR, de interrupt service routine. Quando há a alternância entre os programas, dizemos que há uma troca de contexto. As informações armazenadas nos registradores da CPU precisam ser salvas e restauradas conforme um programa sai e retorna à execução.

Existem quatro causas, ou classes, para as interrupções (Stallings, 2024):

- **E/S:** toda operação de E/S gera uma interrupção, como uma tecla pressionada, um toque na tela, a atualização da imagem no

monitor, um pacote enviado ou recebido na placa de rede, entre outras.

- **Temporização:** qualquer tarefa que utilize tempo utiliza um circuito temporizador no hardware que irá gerar uma interrupção quando o tempo for atingido, de forma análoga a um temporizador de cozinha. Exemplos são o temporizador disparado ao se enviar um segmento TCP através da rede, ou uma instrução que pausa um programa durante cinco segundos enquanto uma mensagem é exibida.
- **Falha de hardware:** um problema de hardware não é um cenário desejado, mas o mecanismo de interrupção permite que se obtenha informações e que se possa tentar uma solução para resolver o problema, em vez de o sistema simplesmente parar de funcionar e precisar ser reiniciado. Isso vale desde algo bem simples como uma impressora sem papel na hora de imprimir, até problemas mais sérios como erros de paridade de memória principal em servidores.
- **Exceção (falha de software):** uma falha na execução de um software gera uma interrupção que recebe o nome específico de exceção. Exemplos são uma tentativa de divisão por zero ou de se executar alguma outra instrução inválida, overflow, ou tentativa de acesso a endereços de memória de outros programas de forma indevida. Esse tipo de interrupção permite que o desenvolvedor possa tentar resolvê-la incluindo uma rotina de tratamento de exceção em seu programa. Por exemplo, um tratamento de exceção para uma divisão por zero pode solicitar que o usuário insira um novo valor que seja válido.

O uso de interrupções viabilizou que o compartilhamento dos recursos fosse “proativo”. O módulo de E/S responsável pelo disco “avisa” a finalização de uma operação por meio da interrupção e o módulo de E/S responsável pelo mouse “avisa” que o usuário clicou no botão.

A implementação de exceções pode conter boa parte dos danos possibilitando correção das falhas, ou viabilizando que apenas o processo com falha tenha que ser encerrado.



IMPORTANTE

O uso de interrupções também é o mecanismo que permite que o SO reassuma o controle do computador. Toda vez que o SO passa o controle da CPU para um processo, ele dispara um temporizador. Se o SO não reassumiu o controle por causa de alguma interrupção ou chamada de sistema, quando o temporizador zerar, será gerada uma interrupção garantindo que o SO reassuma o controle, lembrando que a RTI é parte do kernel do SO.

3 Prática: visualizando informações sobre o hardware e o sistema operacional

Há uma série de comandos que permitem visualizar informações sobre o hardware e o próprio sistema operacional. Lembre-se de que, ao testar os comandos em uma máquina Linux, eles devem ser digitados sem as aspas.

O comando “lscpu” mostra informações sobre o microprocessador. O comando “lsmem” mostra quanto de memória principal há no sistema. Já o comando “lsblk” mostra informações sobre os dispositivos de memória secundária. Execute “lsblk -e7” para uma saída mais limpa.

Os comandos “lsscsi”, “lspci” e “lsusb” irão fornecer informações sobre os dispositivos conectados a esses barramentos mas, caso você esteja utilizando uma máquina virtual em nuvem ou local, eles retornarão pouca ou nenhuma informação.

O comando “lshw” mostra informações completas sobre todo o hardware, mas você precisará executá-lo utilizando “sudo lshw”.

Uma das formas de monitorar a ocorrência de interrupções é através do comando “vmstat -s”. Observe a linha “interrupts” e repita o comando algumas vezes para verificar o valor aumentando.

Passando ao nível do SO, o comando “uname” exibe o kernel, enquanto o “uname -a” inclui a versão, a arquitetura de hardware e outras informações. Em sistemas que utilizam o systemd como gerenciador do espaço de usuário, o comando “hostnamectl” traz informações sobre a máquina, o kernel e a distribuição. Para qualquer distribuição Linux, o comando “cat /etc/*release” fornecerá informações sobre a distribuição.

Ainda sobre distribuições, a mais usada no mercado é a Ubuntu. Ela é derivada do Debian e compartilham algumas ferramentas de configuração e gerenciamento. A RedHat, uma empresa pertencente à IBM tem outra das distribuições mais importantes e é paga. A Fedora é uma versão da RedHat voltada ao usuário final. Já a Amazon AWS e a Oracle mantêm suas próprias distribuições derivadas da RedHat, a Amazon Linux e a Oracle Linux, respectivamente. Outra distribuição muito popular na Europa é a Suse e OpenSuse, sua versão gratuita. Essa lista abrange as principais distribuições em amplo uso no meio corporativo e devem ser o principal foco de quem está estudando o sistema Linux, com exceção de algumas distribuições para finalidades muito específicas, como testes de segurança ou para uso em roteadores.

Considerações finais

Neste capítulo, revisamos alguns conceitos importantes sobre arquitetura de computadores, o funcionamento do hardware, com o foco na atuação do sistema operacional. Não foi um “mergulho profundo”, mas aprendemos como o SO atua para controlar o hardware e como o hardware suporta a atuação do SO. Essa visão é essencial para que

possamos compreender o outro lado da interface do SO com as aplicações. Por fim, lembre-se de praticar os comandos que permitem visualizar os conceitos apresentados.

Referências

MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. 5. ed. Rio de Janeiro: LTC, 2013.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de sistemas operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.

STALLINGS, W. **Arquitetura e organização de computadores**: projetando com foco em desempenho. 11. ed. Porto Alegre: Bookman, 2024.

VALIANTE FILHO, F. **Atuação do sistema operacional sobre o hardware**. 2020. Disponível em: <https://prof.valiante.info/aulas/sistemas-operacionais/atua%C3%A7%C3%A3o-do-sistema-operacional-sobre-o-hardware>. Acesso em: 7 mar. 2025.

Processos e threads: porque os computadores existem para executar programas

O computador existe para executar programas. É verdade que o software não pode ser executado sem o hardware, e o código seria apenas um pedaço de texto, ou, como diz o lema de um software famoso, “código é poesia”. O hardware serviria apenas para ter peso e esquentar quando ligado sem os programas adequados. Provavelmente, há aquecedores mais baratos e eficientes para o frio e pesos de papel ou porta mais baratos e elegantes.

Quando um programa está em execução, ele é chamado de “processo”. É uma definição bastante simples, mas que oculta uma série de atividades que precisam ocorrer. Para mencionar algumas delas:

- O código executável correspondente precisa ser localizado no dispositivo de memória secundária e copiado para um local adequado na memória principal.
- Como há diversos programas em execução no computador, o sistema operacional precisa colocar esse processo na fila para que ele possa ter algumas de suas instruções executadas na CPU em um determinado momento.
- Posteriormente, ele precisará retornar à fila por causa de alguma chamada de sistema, interrupção, ou limite de tempo, e aguardar novamente sua vez de voltar a ser executado até que seja encerrado.

Para gerenciar tudo isso, há uma série de informações necessárias:

- **Process IDentification (PID):** é o número que identifica o processo.
 - Normalmente, os PIDs mais baixos correspondem aos processos do sistema operacional criados na inicialização do sistema. Os processos do usuário final assumem números mais altos. Também é comum que esses números sejam crescentes. O número de processo é o número da execução do programa, ou seja, múltiplas instâncias de um mesmo programa receberão diferentes números de processo.
- **Nome do processo:** enquanto o PID é muito conveniente para o SO, para nós, usuários finais, um nome permite identificar o processo muito mais facilmente. Frequentemente, é usado o nome do arquivo executável.

- **Usuário:** cada processo tem um usuário responsável por ele.
 - O usuário também é identificado por um número de usuário (UID, ou User IDentification), mas é exibido o nome para a conveniência dos usuários.
- **Prioridade:** o SO adota critérios para organizar a fila de processos a serem executados. Alguns são mais importantes do que outros no sistema, mesmo que temporariamente.

Há também uma série de parâmetros que o SO monitora em cada processo, por exemplo: tempo (acumulado) de CPU, uso (proporcional) de CPU, consumo de memória principal, operações de E/S realizadas, uso de rede, entre outros.

Todas essas informações são armazenadas nos chamados blocos de controle de processo (PCB, ou process control block). Há um PCB para cada processo, que fica armazenado em uma área da memória principal reservada para o SO. Nos sistemas Linux, essas informações podem ser visualizadas no pseudossistema de arquivos do diretório “/proc”, que provê “uma interface para as estruturas de dados do kernel” (Linux manpages, 2024).



NA PRÁTICA

Se você está vendo uma versão digital deste material, deve ter um gerenciador de tarefas disponível como utilitário no SO. Abra-o e investigue as diversas informações disponíveis. Verifique as abas, ou seções presentes, as opções para exibir mais dados e as ajudas fornecidas.

Há algumas etapas pelas quais um programa passa durante sua execução, são os chamados estados do processo. Além dos estados de criação e de término, que envolvem respectivamente a definição das informações necessárias para a execução do programa e a coleta de

dados necessária para a sua finalização e liberação dos recursos associados, há outros três estados principais:

- **Execução:** o processo está efetivamente com suas instruções sendo executadas na CPU no momento. O processo somente pode ser finalizado quando está no estado de execução.
- **Pronto:** o processo apenas aguarda sua vez de ser escalonado, isto é, ir para o estado de execução. Após ser criado, o processo passa para o estado de pronto.
- **Espera ou bloqueado:** o processo depende do término de uma operação de E/S ou temporização para que possa voltar para a fila de pronto.

Quando o processo está no estado de execução, ele pode ser finalizado, voltar ao estado de pronto, ou ir para o estado de espera (E/S), ou bloqueado (temporização).

É importante ressaltar que esses estados do processo consideram a escala de tempo da CPU, que executa bilhões de instruções por segundo. Para um observador humano, os processos poderão parecer todos em execução simultânea e, inclusive, a informação constante no gerenciador de tarefas considera essencialmente nossa escala de tempo.

Os processos também podem originar outros processos. Em outras palavras, enquanto um programa está sendo executado, ele pode executar outros programas – seja uma nova cópia de si mesmo, seja programas diferentes. Assim, cria-se uma relação de “pais e filhos”, estabelecendo-se uma hierarquia entre processos. Essa hierarquia é usada no gerenciamento dos processos pelo SO. Quotas de recursos e privilégio de execução são exemplos de características “herdadas” pelos processos filhos. Ao se eliminar um processo pai, os processos filhos também são eliminados, porém, ao se eliminar um processo filho o processo pai permanece. O número do processo pai é identificado como Parent Process ID (PPID) no processo filho.

1 Usuário privilegiado: porque alguém precisa assumir a responsabilidade

Os computadores precisam ser configurados e gerenciados também. Isso significa que ao menos um usuário precisará configurar o SO e definir algumas questões sobre os processos que serão executados naquela máquina, que podem inclusive pertencer a outros usuários. Dessa forma, todo sistema operacional precisa de um usuário privilegiado, para executar essas tarefas. De certa forma, pressupõe-se que esse usuário saiba exatamente o que está fazendo ou, ao menos, assuma a responsabilidade pelo que está fazendo. Nos sistemas Windows, esse usuário é chamado “administrador”, e, nos sistemas Unix e Linux, “root”. Muito embora, nesse último caso, ele também seja referido como “superuser”.

Não é uma boa ideia trabalhar o tempo todo como um usuário privilegiado, já que ele pode fazer qualquer coisa no sistema. Além do risco de segurança, caso a senha desse usuário seja comprometida, há o risco de erros com consequências muito maiores do que as cometidas por um usuário comum. Assim, os sistemas Unix e Linux desenvolveram o mecanismo de `sudo` (superuser do) para poder executar comandos como usuário `root` de forma mais segura (Nemeth *et al.*, 2010; Linux manpages, 2006).

Em geral, o usuário `root` não possui uma senha definida e não pode efetuar login no sistema. Utilizando-se o `sudo`, o usuário que tenha permissão para executar comandos como `sudo` (`sudoer`) solicita ao `root` que execute aquele comando. A senha a ser digitada, ou a chave de criptografia a ser utilizada, é a do usuário. Os comandos deixam registro nos *logs* do sistema. O usuário somente precisa usar o `sudo` quando realmente precisa executar uma tarefa administrativa e não o tempo todo. E fica muito mais fácil definir outro usuário para poder utilizar comandos como o `sudo` em definitivo, ou de forma temporária, sem a necessidade de compartilhamento de senhas. É uma forma de trabalho bem mais segura.

2 Threads

Um programa pode possuir diversos módulos, ou trechos de código, independentes, assíncronos, concorrentes. Considere o exemplo de um editor de texto como o utilizado para a escrita deste material que você está lendo. Há um módulo do programa que cuida da entrada do texto, um segundo, que faz a verificação de ortografia e gramática, um terceiro, que atualiza a tela, e outro para salvar o arquivo em segundo plano. Caso o código executável não tenha nenhuma subdivisão, o salvamento automático iria impedir a entrada do texto, ou a atualização da tela, enquanto não fosse concluída. Mas se esses diversos trechos de código cumprem funcionalidades distintas dentro do todo, eles podem, com alguns cuidados, ser executados de forma independente.

Essas partes independentes do código são justamente as threads, ou linhas de execução. As threads são definidas pelo desenvolvedor e requerem a inserção de instruções adequadas para a sua criação, controle, sincronização e finalização dentro do código. O emprego de threads irá acionar as chamadas de sistema correspondentes, independentemente da linguagem de programação em questão ser compilada ou interpretada.

O exemplo mencionado anteriormente é típico de aplicações em ambiente desktop com interfaces com o usuário final e trechos de código que implementam funcionalidades distintas. Mas há outro caso de uso típico para as threads que busca ganho de desempenho e/ou maior resiliência. É o caso de laços paralelos, divisão de grandes expressões matemáticas em tarefas menores e programas servidores que criam uma thread para o atendimento das requisições feitas por cada cliente.

Nos sistemas Unix, as threads são chamadas lightweight process (LWP). As threads são identificadas com um número de thread, sempre vinculado ao processo. Os recursos, como a quantidade de memória, ou o limite de arquivos abertos, são atribuídos pelo SO ao processo. Mas a

imensa maioria dos sistemas operacionais modernos escalona as threads, de forma que é possível haver diversas threads simultâneas de um mesmo processo em execução, caso haja processadores disponíveis para isso.

À medida que o desempenho aumenta, isso também traz alguns problemas extras para evitar conflitos entre os diversos trechos de código em execução. Por isso, o SO e as linguagens de programação oferecem recursos para a sincronização das threads.

3 Canalização e redirecionamento de processos

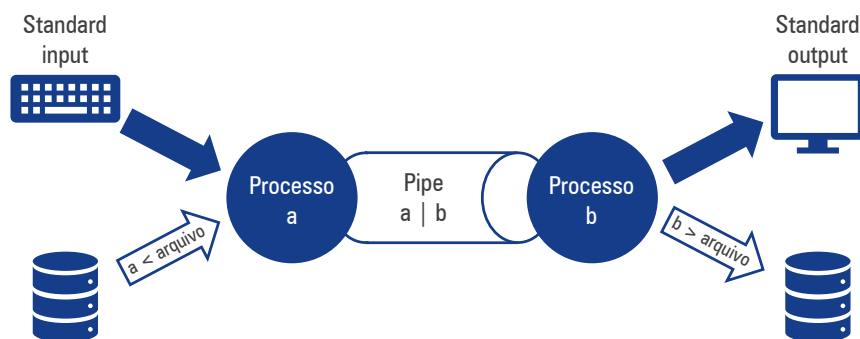
Todo processo possui, associado a ele, um canal de entrada e outro de saída. Na nomenclatura do Unix, eles são chamados de standard input (stdin) e standard output (stdout). No geral, esses canais correspondem a teclado e monitor, ou seja, a entrada do comando é esperada a partir do teclado e a saída será exibida no monitor. Mas é possível redirecioná-los para um arquivo (normalmente), ou para outro dispositivo de E/S. Na maior parte dos sistemas operacionais, esses redirecionamentos são feitos inserindo nos comandos os caracteres "<", para a entrada, e ">", para a saída, além de ">>", que acrescenta a saída do comando no final do arquivo redirecionado.

Além de um redirecionamento simples, é possível conectar o canal de saída de um processo diretamente ao canal de entrada de outro processo por meio de um "pipe". No Unix, essa canalização de processos é representada pelo caractere "|", sendo possível criar canalizações condicionais, em que o segundo processo somente é realizado caso o primeiro tenha sido executado com êxito ("&&"), ou seja, executado apenas se o primeiro houver falhado ("||").

Os processos também possuem um canal de erro, conhecido como standard error (stderr) no Unix. As mensagens de erro são associadas a

esse canal, de forma separada da saída padrão dos comandos, muito embora na tela vejamos os dois canais de saída concatenados.

Figura 1 – Canalização e redirecionamento dos processos



É importante ressaltar que os processos são executados em primeiro ou em segundo plano. Um processo em primeiro plano, ou foreground, é aquele conectado aos canais de entrada e saída padrão, isto é, aqueles que interagem com o usuário; enquanto os processos em segundo plano, ou background, executam sem interação com o usuário, normalmente tendo seus canais associados aos arquivos.

4 Prática: o usuário administrador

Conforme mencionado, em um sistema Unix e Linux, utiliza-se o mecanismo de sudo para a execução de comando como root. Você pode executar “man sudo_root” para ler um pouco mais sobre as vantagens do sudo. Nessa página de ajuda, você pode conferir que, caso precise muito de um terminal como usuário root, deve utilizar “sudo -i” e permanecer assim apenas o tempo estritamente necessário. É um comportamento perigoso trabalhar “logado” como root sem necessidade. Para executar um comando como usuário root, basta digitar sudo, seguido do nome do comando. Dependendo da configuração, é possível que a senha de seu usuário seja solicitada. Por exemplo: “sudo whoami”. Lembre o que faz esse comando e observe a resposta fornecida.

Uma das tarefas de responsabilidade de um usuário privilegiado é justamente poder gerenciar usuários no sistema. Para criar um usuário, use “sudo adduser alunosenac”. Preencha os dados solicitados. Verifique se o usuário foi criado utilizando “lslogins -u” e se foi criado um diretório para esse usuário em /home com “ls /home”.

Agora você pode efetuar login com o novo usuário. Execute alguns comandos e, depois, tente executar um comando com o sudo. Para adicionar esse usuário ao grupo dos “sudoers”, digite “sudo adduser alunosenac sudo”. Faça um novo login com o usuário para receber os novos privilégios. Tente executar um comando utilizando sudo novamente.

Para remover o usuário do grupo de sudo, utilize “sudo deluser alunosenac sudo”, e, para remover completamente o usuário do sistema, use “sudo deluser alunosenac”. Cheque novamente com “lslogins -u” e “ls /home”. Você esperava que o diretório do usuário permanecesse? Por que esse é o comportamento padrão?

Considerações finais

Neste capítulo, abordamos os processos e threads. Compreendemos quais informações serão usadas pelo SO para o gerenciamento dos processos, mas que também são úteis para que o usuário privilegiado possa tomar decisões sobre a administração do sistema.

Um ponto muito importante para se pesquisar e pôr em prática, é verificar como trabalhar com threads nas linguagens de programação que você desenvolve. Pesquise e pratique!

Referências

LINUX manpages. Proc: process information, system information, and sysctl pseudo-filesystem. 2024. Disponível em: <https://man7.org/linux/man-pages/man5/proc.5.html>. Acesso em: 18 mar. 2025.

LINUX manpages. Sudo_root: how to run administrative commands. 2006. Disponível em: https://manpages.ubuntu.com/manpages/noble/man8/sudo_root.8.html. Acesso em: 18 mar. 2025.

NEMETH, E.; SNYDER, G.; HEIN, T; WHALEY, B.; MACKIN, D. **UNIX and Linux System Administration Handbook**. 4. ed. Boston: Prentice Hall, 2010.

Gerenciamento de processos

Em um computador, as CPUs são as responsáveis pela execução dos programas. Então elas precisam receber o trabalho a ser feito. Não podem ficar ociosas e, se houver mais de uma CPU, uma não pode ficar sobrecarregada enquanto outras estão subutilizadas. Se o trabalho for distribuído adequadamente, ele será executado de forma mais eficiente. Definir essa escala de trabalho das CPUs, ou seja, quais processos irão para o estado de execução, é uma tarefa primordial do SO e é chamada de escalonamento, ou scheduling.

Como há concorrência e multitarefa, normalmente há mais processos do que CPUs, ou melhor, mais threads do que CPUs, já que, na maioria dos SOs, o escalonamento é realizado a partir de threads. Com isso, faz-se necessário um revezamento de tempo (time-sharing), em que o uso da CPU é revezado entre várias threads. Uma thread executa um pouco, depois outra, depois outra...

As CPUs modernas executam uma instrução em menos de um bilionésimo de segundo (menos de 1 nanossegundo). Assim, o revezamento é rápido o suficiente para se dar a impressão de tudo estar acontecendo ao mesmo tempo de forma contínua. Um bom exemplo do efeito prático desse truque pode ser observado ao assistir a um filme no cinema com projeção a partir da película. É algo que está ficando menos comum, mas que provavelmente você deve ter tido a experiência. Em geral, são 24 fotos estáticas sendo projetadas por segundo (frames per second) que criam uma perfeita ilusão de movimento contínuo.

No time-sharing dos processos, é atribuída uma “fatia” de tempo (time slice) ou *quantum*. Um limite para o tempo em que a thread pode permanecer no estado de execução antes de ser “convidada a se retirar e ceder a vez” para uma nova thread. Esse é um bom eufemismo para a preempção, pois, com ela, a thread que estava em execução é retornada ao estado de pronto, ou seja, volta para a fila, enquanto outra é escalonada em seu lugar.

Os principais SOs em uso atualmente são preemptivos, pois o SO precisa prevenir que alguma thread não queira mais devolver o controle da CPU, monopolizando recursos indevidamente, comprometendo potencialmente a estabilidade do sistema. Assim, o kernel aciona um temporizador que, se for zerado, dispara uma interrupção de temporização fazendo com que o kernel reassuma o controle. O tempo limite costuma ser bastante generoso em termos computacionais, chegando a milissegundos. É provável que a thread deixe o estado de execução antes em

função de uma operação de E/S, temporização, ou o acionamento de uma chamada de sistema.

As trocas de contexto são implementadas por uma rotina do núcleo chamada dispatcher, enquanto o escalonamento em si é implementado pela rotina do escalonador (scheduler) (Maziero, 2019; Machado; Maia, 2013).

Quando há muitos processos em execução, é provável que haverá alguns processos mais importantes do que outros. Assim, os sistemas operacionais também precisam implementar prioridades. O Windows, por exemplo, implementa 32 níveis de prioridade (Tanenbaum; Bos, 2024) divididas em 6 grupos, de tempo real até a baixa. O Linux emprega um mecanismo de prioridades bastante complexo, com 140 níveis de prioridade, embora normalmente é trabalhado dentro de uma faixa de 40 níveis ajustados por meio de um parâmetro de “grau de gentileza”, o nice-ness, cujos valores variam de -20 (mais prioritário) a +19 (menos prioritário) (Tanenbaum; Bos, 2024; Linux manpages, 2024). Nesses sistemas operacionais, também há preempção por prioridade, de forma que processos mais prioritários fazem com que processos menos prioritários voltem para a fila de pronto. Aliás, esses sistemas apresentam preempção por tempo e prioridade.

O escalonamento precisa de um algoritmo bem projetado para que seja possível o uso eficiente dos recursos e um bom desempenho do sistema. Ele impacta diretamente no tempo de resposta do sistema e na conclusão dos processos. Quando há múltiplas CPUs, o SO pode escalonar diversas threads simultaneamente, mas o escalonamento em si também se torna mais complexo.

1 Os algoritmos de escalonamento dos sistemas operacionais atuais

O escalonamento é a base do gerenciamento de processos, determinando qual processo fará uso da CPU a cada momento. Diversos algoritmos de escalonamento foram desenvolvidos levando em consideração diferentes métricas e considerando o uso de prioridades ou preempção.

Há algoritmos mais simples, como o First In, First Out (FIFO) ou o First Come, First Served (FCFS), que executam os processos na ordem em que chegam, e o Shortest Job First (SJF), que prioriza processos com menor tempo estimado de execução. Incorporando a preempção por prioridades há o escalonamento por prioridades que incorporando a preempção por tempo há uma versão preemptiva do FIFO, chamada escalonamento circular, ou Round-Robin.

O escalonamento circular pode incorporar múltiplas filas de prioridade e implementar ambas as preempções, sendo conhecido como escalonamento circular por prioridades. Ele combina a justiça do Round-Robin, que garante que todas as threads recebam uma fatia de tempo da CPU, com a importância relativa dos processos definidas pelas prioridades, assegurando que tarefas críticas sejam atendidas mais rapidamente.

Para garantir que tarefas de menor prioridade não sejam esquecidas, é necessário implementar um mecanismo de envelhecimento (*aging*) para promover temporariamente a prioridade das threads que estão há mais tempo sem execução.

Para equilibrar processos que tenham maior vínculo, ou afinidade com E/S (I/O-bound), com aqueles que têm maior vínculo com a CPU (CPU-bound), é utilizado um mecanismo de prioridade dinâmica. Processos CPU-bound têm maior chance de serem executados por tempos mais longos.

Já processos I/O-bound, como os processos interativos, isto é, aqueles que executam em primeiro plano e interagem constantemente com o usuário final, têm a tendência de executar apenas uma fração de seu *quantum* e logo irem para o estado de espera ou bloqueio. Com o mecanismo de prioridade dinâmica, o processo pode ser promovido de prioridade após sair do estado de espera para que volte à execução mais rapidamente. Essa promoção normalmente é feita acumulando-se o tempo até completar todo o *quantum* reservado àquele processo.

O escalonamento circular por prioridade implementa as técnicas de *aging* e prioridade dinâmica. Ele é o algoritmo utilizado no Windows.

O Windows 11 incorpora mecanismos de virtualização para maior segurança, com a incorporação de um hipervisor tipo 1, o Hyper-V no nível mais baixo do sistema, o que acrescenta alguma complexidade adicional ao escalonamento (Tanenbaum; Bos, 2024).

O Linux acaba empregando um escalonamento mais complexo, com múltiplos algoritmos de escalonamento conforme a classe de prioridade do processo, conhecido como Multilevel Feedback Queue, ou múltiplas filas com realimentação (Oliveira; Carissimi; Toscani, 2008). Para os processos da classe normal, que corresponde aos quarenta níveis comentados anteriormente, ajustados pelo *niceness*, é usado o algoritmo Completely Fair Scheduler (CFS), ou escalonamento completamente justo, que tem como principal métrica o tempo de CPU atribuído a cada processo, em vez da prioridade e/ou ordem (Tanenbaum; Bos, 2024; Arpaci-Dusseau; Arpaci-Dusseau, 2023).

Nos níveis de prioridade mais altos, são usados algoritmos próprios para sistemas de tempo real com preempção por prioridades nas classes mais prioritárias e preempção por tempo na sequência (Oliveira; Carissimi; Toscani, 2008; Linux manpages, 2024).



PARA SABER MAIS

Para saber mais sobre o CFS, consulte Tanenbaum e Bos (2024), Arpaci-Dusseau e Arpaci-Dusseau (2023), Maziero (2019), além de “CFS Scheduler” na documentação do kernel.

2 Deadlocks: quando as coisas dão errado

O escalonamento aloca o recurso CPU para um processo, mas esse processo, muitas vezes, precisa de algum outro recurso, como um dispositivo de E/S, de armazenamento, ou memória, ou eventualmente precise de mais de um recurso. Mas pode acontecer que um desses recursos esteja alocado a outro processo. Imagine que um processo A tenha alocado um recurso X e precise de Y, enquanto um processo B tenha alocado Y e precise de X. Como processos não têm nenhuma consciência situacional, a grande chance de eles esperarem indefinidamente na “trava da morte” (deadlock) é grande. Se preferir, o termo “impasse” é uma tradução mais técnica nesse caso.

Seres humanos em situações de bom senso resolveriam com um dos processos cedendo o recurso para o próximo, para que depois pudesse usufruí-lo. Bom, isso costuma falhar de vez em quando, em cruzamentos de trânsito em horários de pico, ou com semáforos desligados, ocasionando o mesmo tipo de problema que ocorre em sistemas operacionais. No exemplo do trânsito, algum automóvel irá “sair por outro lado” para poder liberar o trânsito. No caso do sistema operacional, o kernel será obrigado a eliminar processos até que o sistema saia do impasse.

Para entender os deadlocks, é preciso considerar conceitos como “exclusão mútua”, que preconiza que apenas um processo pode acessar um recurso por vez, “seção crítica”, ou “trecho de código”, que acessa os recursos compartilhados. Para tentar evitá-los, há mecanismos como

semáforos, que são variáveis utilizadas para controlar o acesso a recursos compartilhados, assim como suas versões nas ruas, que evitam impasses mais frequentes no trânsito. Os sistemas de mensageria permitem que os processos se comuniquem de forma indireta, por meio de uma caixa de mensagens, ou buffer, e assim coordenem o acesso aos recursos de forma mais flexível.

3 Prática: monitoramento e gerenciamento de processos

O Linux oferece um amplo conjunto de ferramentas para monitorar e gerenciar processos. Os comandos “top” e “htop” permitem visualizar, de forma dinâmica, informações detalhadas sobre os processos em execução, incluindo o PID, uso de CPU, memória, entre outros. Também é possível gerenciar esses processos de forma dinâmica ajustando a prioridade, ou enviando sinais, inclusive o de término. O “htop” tem uma interface mais interativa e intuitiva, com os comandos bastante claros por meio das teclas de função. Para descobrir as opções do “top”, use a tecla “h” para obter ajuda, e a tecla “q”, para sair.

O comando “ps” fornece um “instantâneo” dos processos (process snapshot), sendo muito útil para ser incorporado a scripts, ou obter informações de processos específicos. Aliás, a busca por um processo pode ficar bastante facilitada com o comando “pgrep”, enquanto “pstree” permite visualizar a árvore de processos (processos pais, filhos, etc.).

Já as threads são referenciadas como lightweight process (LWP) e podem ser facilmente visualizadas no “htop”. Para visualizar as threads com o “ps” tente “ps m”, “ps -Lf” e “ps -eLf”.

O comando “kill” envia um sinal para um processo. Mas ele costuma ter um efeito prático mais próximo do significado literal de seu nome, pois, por padrão, é enviado o sinal de encerramento do processo.

Para executar um processo em segundo plano, acrescente um “&” logo após o comando. Por exemplo, “htop &”. Para trazer o programa novamente ao primeiro plano, use o comando “fg”. Para levar o processo ao segundo plano novamente, use a combinação de teclas <Ctrl> + <Z>. Investigue o comando “jobs” para saber como lidar com mais de um processo em segundo plano ao mesmo tempo.

É possível ajustar a prioridade de execução dos processos com os comandos “nice”, para executar um processo com a prioridade diferente da prioridade de execução padrão, e “renice”, para alterar a prioridade de um processo já em execução. Eles sempre trabalham com o *niceness*, que aparece nos comandos de gerenciamento de processos como “NI”.

Os caracteres “<”, “>” e “>>” atuam como redirecionadores, além do comando “tee”, conforme apresentado anteriormente. O pipe “|” canaliza processos, enquanto as combinações “||” e “&&” estabelecem canalizações condicionais. Em Valiante Filho (2023), há diversos exemplos de uso.

Considerações finais

O gerenciamento de processos é um aspecto fundamental de qualquer sistema operacional moderno. Ele impacta diretamente a performance, a estabilidade e a responsividade do sistema. Os algoritmos de escalonamento dos sistemas atuais acabam sendo bastante elaborados e implementam prioridades e ajustes de tempo, além de utilizar preempção.

Para um desenvolvedor de software, saber implementar mecanismos como semáforos e sistemas de mensageria é essencial em projetos de software de maior porte. Aproveite para investigar como esses mecanismos são executados nas linguagens de programação que você mais utiliza.

Para além de comprovar os conceitos teóricos estudados, saber monitorar e gerenciar processos em um servidor é uma competência essencial para qualquer profissional de TI, o que mostra que a prática é essencial.

Referências

ARPACI-DUSSEAU, R. H.; ARPACI-DUSSEAU, A. C. **Operating Systems**: Three Easy Pieces. Versão 1.10. Madison: Arpaci-Dusseau Books, 2023. Disponível em: <https://pages.cs.wisc.edu/~remzi/OSTEP/>. Acesso em: 30 mar. 2025.

CFS Scheduler – Linux Kernel Documentation. Disponível em: <https://www.kernel.org/doc/Documentation/scheduler/sched-design-CFS.txt>. Acesso em: 30 mar. 2025.

LINUX manpages. Sched: overview of CPU scheduling. 2024. Disponível em: <https://man7.org/linux/man-pages/man7/sched.7.html>. Acesso em: 29 jan. 2025.

MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. 5. ed. Rio de Janeiro: LTC, 2013.

MAZIERO, C. A. **Sistemas operacionais**: conceitos e mecanismos [recurso eletrônico]. Curitiba: DINF; UFPR, 2019. Disponível em: <http://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start>. Acesso em: 29 jan. 2025.

OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S. S. **Sistemas operacionais**. 3. ed. Porto Alegre: Bookman, 2008. (Série livros didáticos do instituto de informática da UFRGS, v. 11).

TANENBAUM, A. S.; BOS, H. **Sistemas operacionais modernos**. 5. ed. Porto Alegre: Bookman, 2024.

VALIANTE FILHO, F. **Nanocurso de Linux**. 2023. Disponível em: <https://prof.valiante.info/aulas/nanocurso-de-linux-especial>. Acesso em: 30 mar. 2025.

Gerenciamento de memória

Para que haja múltiplos processos em execução em um computador, esses códigos executáveis devem estar todos carregados na memória principal, bem como os dados manipulados por eles. Em outras palavras, a concorrência inclui a memória principal. Um recurso finito, que precisa ser compartilhado de maneira eficiente, segura e organizada para garantir um bom desempenho do sistema.

Por “segurança”, entenda que um programa não deve invadir a área de memória do outro e nem ler dados de outro programa, mas também

pode ser que tenha uma área compartilhada, ou compartilhe bibliotecas e outras partes da memória com outros processos.

Assim, a memória principal precisou ser dividida em partições. Cada partição poderia conter um processo, permitindo que vários processos residissem ao mesmo tempo na memória. Mas essa divisão não era trivial. A escolha do tamanho das partições poderia levar a problemas como fragmentação interna e fragmentação externa. Na fragmentação interna, havia espaço desperdiçado dentro de uma partição alocada. Por exemplo, com uma partição de 10 k tendo um programa de 5 k alocado. O espaço vazio era desperdiçado. Já na fragmentação externa, havia espaço livre suficiente na memória, no total, mas não de forma contígua para permitir alocar um novo processo. Ou seja, o espaço era desperdiçado porque havia memória livre, mas nenhum espaço suficiente para que o novo código coubesse na memória.

No início, os programas eram compilados para um endereçamento de partição específica. Com a evolução dos sistemas computacionais e dos sistemas operacionais (hardware e software evoluindo juntos), ele passou a poder ser carregado em qualquer endereçamento disponível na memória usando o recurso de relocação. Com esse recurso os programas eram compilados para um endereço inicial e esses endereços eram ajustados ao carregá-lo na memória para corresponder ao endereço real. Posteriormente, foi possível trocar o endereçamento de um programa já carregado para poder eliminar a fragmentação e aceitar novos processos e dados na memória. Todos os processos eram relocados no começo, ou no fim, da memória para deixar o espaço livre contínuo e isso monopolizava o sistema enquanto era feito.

É interessante notar que os processos não podiam ser “quebrados” na memória. Por um lado, eles tinham que estar carregados inteiros, ou pelo menos com seus módulos inteiros conforme a técnica empregada para o desenvolvimento. Por outro lado, isso não era um problema para o hardware, pois a memória de acesso aleatório (RAM) tem esse nome

justamente porque não importa qual a sequência de endereços a ser acessada. O tempo e o procedimento para acessar duas vezes uma mesma posição são o mesmo para uma posição e a seguinte, ou para passar da primeira para a última posição. Era necessária uma nova técnica para gerenciar a memória: a memória virtual.

1 Memória virtual

Frequentemente, na computação, quando algo está muito complexo, quando se precisa subir o nível de abstração, recorre-se à virtualização. A virtualização dissocia o lógico do físico, permitindo que se trabalhe em um domínio mais lógico, muito embora, obviamente, alguém precise lidar com a complexidade, com a conexão com o físico que foi devidamente abstraída.

No caso da memória virtual, a ideia principal é criar uma abstração entre a visão lógica da memória, isto é, como cada programa a enxerga, e a memória física real disponível. Essa dissociação permite que os processos enxerguem (e acessem) uma memória muito grande, maior do que realmente poderiam usar, e ainda exclusiva.

Resolver a complexidade por trás da tradução entre os endereços lógicos e físicos é responsabilidade do sistema operacional, contando com o apoio da unidade de gerenciamento de memória no hardware, a Memory Management Unit (MMU). O SO utiliza as técnicas de paginação ou segmentação para realizar essa tarefa, sendo a paginação a forma empregada na maioria dos SOs atuais, inclusive o Linux e o Windows.

A memória virtual por paginação é uma das técnicas mais utilizadas atualmente. Ela divide tanto a memória lógica (do programa) quanto a memória física em blocos de tamanho fixo chamadas páginas e frames, respectivamente. O sistema operacional mantém uma tabela de páginas que mapeia as páginas lógicas para os frames físicos. Quando um programa acessa um endereço lógico, o sistema operacional utiliza a tabela

de páginas para encontrar o frame físico correspondente e acessar os dados na memória física. Essa tradução é realizada pela MMU.

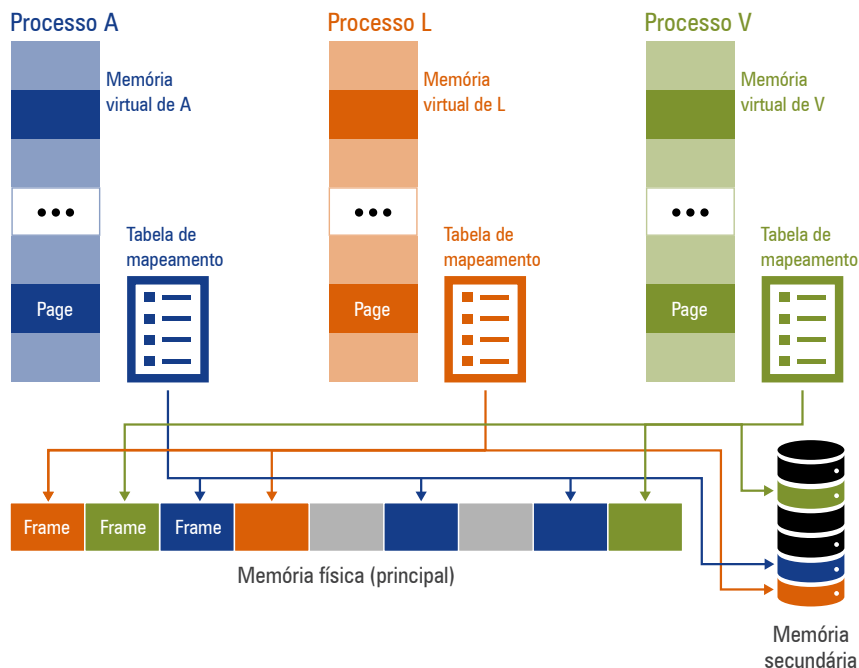
Detalhando um pouco mais: quando um programa é executado, o kernel carrega o começo do código, ou seja, as primeiras páginas virtuais, na memória principal, em páginas reais, que são os frames. A memória virtual permite que o código não esteja todo carregado na memória física. Ele está inteiro na memória principal, exclusiva para ele, mas precisará de fato que o trecho em execução no momento esteja na memória física para que isso aconteça. Se chegou a um trecho de código que não está disponível na memória física, o kernel terá que carregar essa página. Essa técnica é chamada de paginação sob demanda (on-demand paging). O kernel também irá ajustar a quantidade de páginas para a memória de trabalho do processo (Ward, 2015; Machado; Maia, 2013).

Quando a página virtual não está disponível na memória principal, ocorre uma falha de página, ou page fault. Isso ocasiona uma interrupção de E/S e a página precisará ser carregada (Machado; Maia, 2013). Os sistemas Linux distinguem dois tipos de page fault: minor e major (Ward, 2015; Godard, 2025; Linux manpages, 2025). As minor page faults não costumam causar grande impacto, pois envolvem basicamente as informações de mapeamento, sem envolver acesso ao dispositivo de memória secundária. Já nas major page faults, é necessário buscar a página na memória secundária. É natural que isso ocorra conforme o programa vai sendo executado e carregado na memória, mas pode se tornar um problema se isso ocorre porque há pouco espaço disponível e é necessário trocar (swap) páginas da memória principal com a secundária para criar espaço.

O kernel ajusta a quantidade de páginas virtuais que serão mapeadas em páginas reais (frames) na memória para cada processo, o que é chamado de conjunto de trabalho (working set).

A figura 1 ilustra a relação entre a memória virtual dos processos, seu mapeamento na memória física e as partes na memória secundária.

Figura 1 – Esquema de funcionamento da memória virtual



PARA SABER MAIS

Há muitos detalhes adicionais que poderiam ser mais bem aprofundados neste capítulo, mas simplesmente estamos ficando “sem espaço livre”. Por exemplo: de que forma o SO aloca os espaços na memória? Como ele gerencia os espaços livres? Como substitui frames na memória física? Como controla o que foi modificado? Como definir o conjunto de trabalho ideal? Detalhes do mapeamento, TLBs, etc. Nossas respostas estão “armazenadas em swap” no capítulo 3 de *Sistemas operacionais modernos*, livro de Tanenbaum e Bos (2024), e nos capítulos 9 e 10, de *Arquitetura de sistemas operacionais*, livro de Machado e Maia (2013).

2 Quando não há memória principal suficiente

Nas situações em que a demanda por memória excede a capacidade da memória principal, o SO precisa recorrer a técnicas para estender a memória disponível. Uma possível solução é utilizar a memória secundária (disco rígido ou SSD) como uma extensão da memória principal através de um mecanismo de troca (swap). O swap consiste em mover temporariamente páginas da memória que não estão em uso para a memória secundária (page out), liberando frames na memória principal para os outros processos. Quando um programa tenta acessar uma página que foi trocada para o disco, ocorre uma falha de página, e o sistema operacional precisa buscar a página no disco e trazê-la de volta para a memória principal (page in).

No Windows, esse espaço em disco utilizado para a troca é chamado “arquivo de paginação” (pagefile), enquanto no Unix e no Linux é “arquivo de swap” (swapfile), ou uma partição específica para isso. O uso desse mecanismo permite resolver o problema da quantidade de memória disponível (tamanho), mas cobra um preço alto no desempenho geral do sistema uma vez que, mesmo utilizando SSDs, a memória secundária é muito mais lenta do que a principal.

Atualmente, com o aumento da capacidade da memória principal em computadores pessoais e servidores, a necessidade de utilizar o swap diminuiu consideravelmente. Muitos sistemas Linux já não apresentam o recurso habilitado por padrão. No entanto, o Android voltou a utilizar esse mecanismo em dispositivos móveis. Habilitar ou não o mecanismo de troca e gerenciar sua utilização pode ter um grande impacto no desempenho do sistema.

Se não houver mecanismo de troca, ou se mesmo assim o sistema estiver criticamente sem memória o SO se verá obrigado a liberar espaço

de maneira abrupta e forçada em nome da estabilidade do sistema, eliminando alguns processos. A ideia é que é melhor manter o sistema funcionando com alguns processos a menos que poderão ser reiniciados, do que o sistema inteiro reiniciar. O kernel Linux chama isso de Out of Memory killer (OOM killer). O nome parece saído de um filme de faroeste e a descrição é bem condizente: “o OOM *killer* seleciona uma tarefa para sacrificar em consideração a saúde geral do sistema. A tarefa selecionada é morta na esperança de que após sua partida memória suficiente seja liberada para continuar a operação normal” (The kernel..., 2025, tradução nossa). Felizmente, após essa descrição trágica, sabemos que nenhuma gota de sangue real será derramada por esse matador. Mas ainda poderá provocar muitas gotas de suor real no administrador do sistema.

3 Prática: monitoramento da memória e gerenciamento de pacotes

Para monitorar o uso da memória em sistemas Unix e Linux, o comando “free” é essencial. Experimente “free -ht”. Ele exibe informações sobre a quantidade total de memória, quantidade utilizada e livre (sem uso). A coluna “buffers/cache” tem relação com o uso da memória secundária. As informações na memória principal a serem escritas no dispositivo de memória secundária são colocadas em buffer, enquanto aquelas que foram lidas do dispositivo são armazenadas em cache na memória principal (Ward, 2015). Mas talvez a coluna mais importante a ser observada seja a de memória disponível (available), pois é ela que indica o quanto o SO ainda pode alocar para novos processos. O comando “lsmem” lista a memória física instalada. Os comandos que gerenciam processos de forma dinâmica como o “top” e o “htop” também trazem informações sobre o uso da memória principal.

Outros comandos que servem para monitorar o desempenho da memória principal e o uso do mecanismo de swap são: “vmstat” e

“pidstat”. O uso de memória de um processo pode ser investigado com “pmap”. Visualize a página de ajuda desses comandos para saber como utilizá-los e, principalmente, como interpretar as saídas fornecidas.

Investigue como habilitar e inibir o swap e como ajustar sua utilização em sistemas Linux.

Outro tópico muito importante na administração de sistemas Unix e Linux é o gerenciamento de pacotes (programas). O gerenciamento de pacotes permite atualizar o sistema (upgrade), instalar programas, remover programas, entre outros. As ferramentas de gerenciamento de pacotes são um dos aspectos mais perceptíveis de diferenças entre as diferentes distribuições Linux e entre diferentes Unixes. As distribuições Ubuntu, Debian e outras derivadas utilizam a ferramenta e o respectivo comando “apt”. Distribuições embasadas na Red Hat, que incluem Amazon Linux, Oracle Linux e Fedora, utilizam o “dnf”. Distribuições baseadas no Suse usam o “zypper” e assim por diante. É fundamental se familiarizar com o gerenciador de pacotes da distribuição em uso, embora a mesma lógica esteja presente em todos eles:

- É feita uma consulta aos repositórios de software da distribuição para se obter a lista mais atualizada dos pacotes.
- Para atualizar o sistema, a ferramenta verifica quais os pacotes mais recentes em relação aos que já estão instalados, bem como as dependências necessárias.
- Para se instalar um novo programa, a ferramenta também verifica suas dependências, isto é, que outros programas ou bibliotecas também são necessários e em quais versões, providenciando para que sejam instalados ou atualizados em conjunto.
- Os pacotes dos programas são finalmente baixados, “desempacotados”, e os arquivos são copiados em seus respectivos destinos. Eventualmente, os arquivos de documentação, como as

manpages, também são copiados e a base de dados do manual é atualizada;

- Para remover um programa, basta remover os arquivos. Simples assim!

Nos sistemas Ubuntu (e Debian), como abordamos em nossas seções práticas, a atualização do sistema pode ser feita utilizando-se os comandos “sudo apt update” para atualizar os repositórios e “sudo apt upgrade” para a atualização em si. Preste atenção nas informações fornecidas nas saídas dos comandos. As opções “autoclean” e “autoremove” também podem ser usadas para eliminar arquivos desnecessários.

Já para instalar um programa, usa-se “sudo apt install nomedoprograma” e para removê-lo “sudo apt remove nomedoprograma”. Se precisar de uma sugestão de programa simples para testar, use o “sl”.

Como há uma diversidade de formatos de pacotes, recentemente, as principais distribuições têm se utilizado de containers para a distribuição de aplicações que buscam aumentar a segurança, resolver eventuais dependências conflitantes e possibilitar que um único formato possa ser usado em todas as principais distribuições. Assim, vale a pena conhecer os formatos Snapcraft e Flatpak que têm se tornado cada vez mais populares.

Considerações finais

Neste capítulo, exploramos os conceitos fundamentais do gerenciamento da memória principal, desde o particionamento de memória física até a memória virtual. É um tema bastante complexo e apenas “arranhamos a superfície” dele, mas é um conhecimento essencial para tomar decisões informadas sobre a alocação de recursos e a configuração do ambiente computacional. Com certeza, esse é um tópico que vale a pena ser explorado com atenção. Para isso, consulte a bibliografia sugerida,

pratique os comandos com atenção e pesquise os tópicos mencionados ao longo do texto.

Referências

GODARD, S. **pidstat(1)**: Linux manual page. 2025. Disponível em: <https://man7.org/linux/man-pages/man1/pidstat.1.html>. Acesso em: 7 abr. 2025.

LINUX manpages. Top: display Linux processes. 2024. Disponível em: <https://man7.org/linux/man-pages/man1/top.1.html>. Acesso em: 7 abr. 2025.

MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. 5. ed. Rio de Janeiro: LTC, 2013.

TANENBAUM, A. S.; BOS, H. **Sistemas operacionais modernos**. 5. ed. Porto Alegre: Bookman, 2024.

THE KERNEL DEVELOPMENT COMMUNITY. **The Linux kernel user's and administrator's guide**. 2025. Disponível em: <https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html>. Acesso em: 7 abr. 2025.

WARD, B. **Como o Linux Funciona**: o que todo superusuário deveria saber. São Paulo: Novatec, 2015.

Gerenciamento de sistema de arquivos

Imagine-se em uma grande biblioteca. Milhares de livros espalhados por diversas prateleiras, estantes, corredores e, em algumas bibliotecas, até por diversos andares. Sem uma organização precisa, seria impossível encontrar um livro específico. A biblioteca precisa de um sistema: um catálogo, ou índice, que indica a localização de cada livro a partir do título, autor ou assunto; estantes e prateleiras organizadas por áreas do conhecimento e temas com a devida identificação; regras para adicionar e remover itens, entre outros.

O gerenciamento de arquivos em um computador é análogo a essa biblioteca. Assim como uma biblioteca armazena informações em livros, um computador armazena dados em dispositivos de memória secundária, como drives de estado sólido (SSDs), cartões de memória, e pen drives. Nesse caso, os dados podem estar espalhados fisicamente por toda a mídia de armazenamento, e não necessariamente em trechos contíguos.

Um sistema de arquivos fornece uma abstração do hardware físico, permitindo que se trabalhe com arquivos e pastas de forma lógica (simples), sem se preocupar com a exata localização física dos dados (complexo). Ele também é responsável por recuperar e reunir todas essas partes para recompor os dados. Dessa forma, independentemente do dispositivo ou padrão de barramento usado (USB, SATA, SAS, etc.), é possível trabalhar com diretórios e arquivos de maneira uniforme, prática e flexível, seja como usuário final, seja como desenvolvedor. É possível criar, excluir, copiar, mover e renomear arquivos e diretórios, além de abrir, ler, modificar (editar) e escrever (gravar) os arquivos. Todas essas operações possuem as respectivas chamadas “sistema para o kernel do SO”.

Retomando nosso exercício de imaginação, cada livro (arquivo) precisa estar localizado em estantes (dispositivos de armazenamento, ou volumes lógicos) organizadas em seções temáticas (pastas ou diretórios). Um índice permite que a bibliotecária (o sistema operacional) localize rapidamente o livro desejado e o entregue ao usuário (programa). Sem um índice eficiente, encontrar um livro na biblioteca seria muito difícil e ineficiente, e seria impossível encontrar um conjunto de dados que constitui um arquivo. Olhando mais atentamente um livro (arquivo), veremos que ele será composto de pequenos conjuntos de folhas costuradas ou coladas, chamadas cadernos (blocos), que depois são encadernados.

Um arquivo é essencialmente um conjunto de dados organizados que recebe um nome e possui outros atributos, ou metadados, associados,

tais como proprietário, data e hora de criação e modificação, tamanho, mecanismos de proteção, entre outros. Eles também podem ser de diversos tipos como executáveis, de texto, de imagem, etc. Os arquivos são organizados em uma estrutura lógica hierárquica em forma de árvore, composta por diretórios (ou pastas) e subdiretórios (ou subpastas).

Assim, é possível representar um arquivo de forma unívoca por meio de seu caminho (path) completo na hierarquia, ou árvore de diretórios. Por exemplo, o arquivo `"/home/estudante/aula_so/notas_de_aula.txt"` encontra-se no terceiro nível hierárquico a partir da raiz da árvore de diretórios do sistema de arquivos (`/`), sendo `"/home"` o primeiro nível, `"/home/estudante"` o segundo nível, que identifica o usuário nesse exemplo, e `"/home/estudante/aula_so"` o subdiretório no terceiro nível em que está o arquivo `"notas_de_aula.txt"`. Pode haver outros arquivos com esse nome no sistema, mas em diferentes caminhos. Note que o caminho apresentado está usando a nomenclatura dos sistemas Unix (e Linux) e recorde-se do Linux FHS. Em um sistema Windows, o arquivo poderia estar em um endereço como `"C:\Users\Estudante\Documents\aula_so\notas_de_aula.txt"`. O sistema de arquivos mantém um índice de toda essa estrutura e os metadados associados. Alguns exemplos de sistemas de arquivos são o EXT, de EXTendend file fystem, muito usado no Linux, e o NT File System (NTFS), do Microsoft Windows (EMC, 2015).

Apesar de a analogia da biblioteca ser perfeita para compreendermos o gerenciamento do sistema de arquivos, o nome vem de uma analogia mais direta, com o armazenamento de informações físicas em escritórios.

Vamos retomar a ideia dos blocos que compõem os arquivos. Eles são a menor unidade de armazenamento para os dados no sistema de arquivos, definidos em sua criação (formatação), enquanto, na mídia física, isso corresponde ao setor. Isso significa que toda operação de leitura e escrita é feita com o tamanho de um bloco. Esse também é o tamanho mínimo de um arquivo e, se o arquivo tiver mais do que esse tamanho, ele será dividido em vários blocos. Como exemplo, um tamanho de bloco

muito comum nos sistemas de arquivos atuais é de 4 KB. Em sistemas Unix e Linux, os discos também são chamados de dispositivos de bloco.

Se o bloco é a menor unidade lógica, no outro extremo, a situação é mais abstrata. Os sistemas de arquivos normalmente são alocados em volumes lógicos. Esses volumes lógicos precisam ser alocados em volumes físicos como um ou mais SSDs, por exemplo. Quando um volume físico é dividido em mais de um volume lógico, isso é chamado de particionamento e cada divisão é chamada de partição. Essa técnica é muito comum em computadores pessoais. Porém, em servidores, é muito comum agrupar diversos volumes físicos de forma a serem vistos como um único volume lógico, por meio da técnica de concatenação (EMC, 2015).

O sistema de arquivos precisa manter um índice mapeando os arquivos aos blocos correspondentes. Como os arquivos normalmente estão espalhados por vários blocos que podem estar espalhados ao longo do disco, foi adotado um esquema de blocos de índice, index-nodes, ou simplesmente inode. O índice aponta para o inode do arquivo, que aponta para os blocos que constituem aquele arquivo. O índice do sistema de arquivos também precisa organizar os blocos livres e os atributos dos arquivos. Ao se formatar um volume lógico, uma das principais tarefas envolvidas é justamente a criação desse índice.

Entre os atributos, há os mecanismos de proteção de acesso (segurança) aos arquivos, para delimitar quem pode fazer o que com cada arquivo e diretório. Nos sistemas Unix e Linux, isso é feito por meio de bits de permissão para leitura (r – read), escrita (w – write), que abrange a modificação e a exclusão, e execução (x – eXecute) para o proprietário do arquivo (u – ser), o grupo a que pertence o proprietário (g – group) e os outros usuários (o – others). É um mecanismo simples e eficiente, mas até certo ponto engessado, pois não é possível determinar que um usuário específico dentro de um grupo tenha uma permissão diferente. Outra técnica mais granular e flexível, mas também mais complexa para se administrar e implementar, é embasada em listas de controle de

acesso, ou Access Control List (ACL) em inglês, que é o mecanismo padrão do Windows.

Outra característica importante dos sistemas de arquivos modernos é o *journaling*, que registra as alterações no sistema de arquivos antes que estas sejam realmente aplicadas ao disco. Isso garante a consistência dos dados em caso de falhas inesperadas, como quedas de energia ou travamentos. Se ocorrer uma falha durante uma operação de gravação, o sistema operacional pode usar o *journal* para reverter ou completar a operação, evitando a corrupção do sistema de arquivos.

Finalmente, temos a variável de ambiente “PATH”, que contém uma lista de diretórios em que o sistema operacional procura o arquivo executável quando você digita um comando no terminal, ou clica/toca na interface gráfica. O sistema vai procurar nos diretórios listados no PATH até encontrar o executável correspondente, ou, caso não encontre, emitirá uma mensagem de erro informando que o comando não foi encontrado. A variável de ambiente PATH facilita a execução dos programas ao evitar que se precise especificar o caminho completo cada vez que são invocados. Para verificar essa variável de ambiente, utilize o comando adequado para o interpretador de comandos em uso: “echo \$PATH” no Bash, ou “\$env:PATH” no PowerShell.



PARA SABER MAIS

Há muito mais assuntos que podem ser aprofundados. Conforme seus interesses e necessidades, você talvez queira se aprofundar em tópicos relacionados à segurança e à disponibilidade dos dados armazenados, como a técnica Redundant Array of Independent Disks (RAID), backup e arquivamento; técnicas de otimização de armazenamento, como compressão e deduplicação; sistemas de arquivos em rede e distribuídos; object-based storage para armazenamento de dados em nuvem e aplicações de big data.

1 Prática: manipulando o sistema de arquivos

Em um sistema operacional Unix, ou compatível, o sistema de arquivos também possui “pontos de interação”, ou representações, com dispositivos de E/S, processos, estruturas de dados do kernel e muito mais. Nesse sentido, todos são legíveis e eventualmente manipuláveis de maneira uniforme como se fossem arquivos, embora efetivamente não sejam.

Ao utilizar a opção detalhada do comando para a listagem de arquivos com “ls -l”, o primeiro caractere revela do que se trata aquela entrada no sistema de arquivos em questão. Os mais comuns são “-” para um arquivo, ou algo do SO que se faça parecer com um arquivo, “d” para um diretório e “l” para um link. Mas os próprios discos também são representados por um “b”, de dispositivos de bloco (block devices).

Os próximos nove caracteres correspondem às permissões de leitura (r), escrita (w) e execução (x) para o usuário (u), grupo (g) e outros (o). Um arquivo que apresente as permissões “rw-r--r-” significa que pode ser lido e editado pelo usuário proprietário do arquivo, apenas lido pelos demais usuários no grupo e apenas lido por qualquer outro usuário. Já um arquivo com as permissões “rwxr-xr-x” tem acesso total por parte do usuário, enquanto o grupo e os outros usuários podem ler e executar o arquivo.

O comando “chmod” manipula as permissões e é muito comum usar a notação decimal (a rigor octal) equivalente ao binário correspondente de cada grupo conforme os parâmetros ligados. Assim, “rw-r--r-” (110 100 100) é 644, enquanto “rwxr-xr-x” (111 101 101) é 755. Para ajustar os parâmetros dessa forma, seria possível utilizar “chmod 644 arquivo” e “chmod 755 arquivo”.

Também é possível combinar os caracteres “u”, “g” e “o” com “r”, “w” e “x” no comando. Por exemplo, “`chmod +x outroarquivo`” torna-o executável para todos, enquanto “`chmod u+x outroarquivo`” torna-o executável apenas para o usuário. Já “`chmod g-w outroarquivo`” retira a permissão de escrita para os membros do grupo.

Caso você se depare com um “s” no lugar do “x” no bloco de controle do usuário, isso significa que o programa será executado com o usuário do proprietário do arquivo, e não com o usuário atual. Esse é um truque útil para permitir que um usuário comum execute um programa como root, por exemplo, e é chamado bit SETUID (Tanenbaum; Bos, 2024). Caso não existisse esse mecanismo, um usuário comum não poderia trocar sua própria senha, pois isso requer acesso privilegiado, entre outras ações. Para habilitar o bit de SETUID em um arquivo, é necessário usar o número 4 como um quarto número no `chmod`, ou usar a opção “=s” ao utilizar as letras. A permissão do arquivo executável do comando “`passwd`” (`/usr/bin/passwd`) é “`rwsr-xr-x`”. Ela pode ser obtida com “`chmod 4755 arquivo`”, ou “`chmod u=srwx arquivo`” (Nemeth, 2010).

Agora, vamos explorar alguns comandos básicos para manipular o sistema de arquivos em sistemas Unix e Linux.

Diretórios:

- **mkdir:** cria um novo diretório com o nome especificado. Exemplo: “`mkdir novodir`”.
- **rmdir:** remove um diretório vazio. Exemplo: “`rmdir novodir`”.

Arquivos:

- **touch:** cria um arquivo vazio. Exemplo: “`touch novoarquivo`”. Touch também pode ser usado para atualizar a data de modificação de um arquivo existente.
- **cp:** copia arquivos ou diretórios. Exemplo: “`cp novoarquivo umacopia`”.

- **mv:** move arquivos. Exemplo: "mv arquivo diretorio/arquivo".
 - Caso o diretório de destino seja o mesmo diretório atual, o comando mv irá renomear o arquivo. Exemplo: "mv arquivo novo_nome".
- **rm:** remove um arquivo e diretórios com conteúdo. Tenha cuidado ao usar esse comando, pois a exclusão é irreversível. Se quiser, use a opção "-i" para confirmar antes de remover cada arquivo. Exemplo: "rm arquivo".
 - A opção "-R", ou "-r", remove recursivamente e é usada para remover diretórios com seus respectivos conteúdos. Dica importante: use sempre "ls -R" antes de usar "rm -R", assim você saberá exatamente o que será excluído.

Expansão e globbing:

- O interpretador de comandos suporta o uso de caracteres curinga (wildcards) para facilitar a manipulação de múltiplos arquivos. O asterisco (*) representa qualquer sequência de caracteres, enquanto o ponto de interrogação (?) representa um único caractere. Exemplo: "rm *.txt" remove todos os arquivos com extensão ".txt" no diretório atual.
- As chaves podem ser usadas para expandir termos. Exemplo: "touch arq{0..9}" criará os arquivos "arq0", "arq1", até "arq9", e "touch arq.{txt,md}" criará os arquivos "arq.txt" e "arq.md".
- Os colchetes permitem selecionar caracteres ou sequências de caracteres. Exemplo: "ls arq[3-6]" listará apenas os arquivos "arq3", "arq4", "arq5" e "arq6" da lista criada no parágrafo anterior.



IMPORTANTE

As extensões para os arquivos não são obrigatórias nos sistemas Unix e Linux! De fato, com exceção daquelas extensões que permitem ao usuário final identificar o tipo de arquivo, como “pdf”, “txt”, “jpg” e assim por diante, sendo muito raro haver extensões. Quando ocorrem, elas normalmente são meros informativos como “.backup”, “.old”, ou “.config”. Use o comando “file” para verificar o tipo de arquivo e para saber mais.

Para criar e editar arquivos de texto diretamente na linha de comando, podemos usar editores como o “vi” (ou “vim”) e o “nano”. O “vi” é um editor poderoso, mas com uma curva de aprendizado mais acentuada. Isso é só um eufemismo para dizer que ele é muito difícil de usar, sendo a causa de muitos memes e pegadinhas. O “nano” é um editor mais amigável, ou, se preferir, mais humano.

No “vi”, crie ou abra um arquivo digitando “vi nomedoarquivo”. Para poder editar, você deve pressionar a tecla <i> (sim, a letra “i” mesmo). Para acessar o menu, primeiro pressione <ESC> e depois <:>. Digite “w” e <Enter> se quiser escrever o arquivo (salvar), “wq”, para salvar e sair, ou “q!”, para sair sem salvar.

No nano, crie ou abra um arquivo digitando “nano nomedoarquivo”. Edite o conteúdo normalmente, sem precisar de nenhum truque. Observe as opções do menu na parte inferior da tela. O “^” remete à tecla “Ctrl” (control). Ou seja: use <Ctrl>+<O> para salvar (escrever) o arquivo, e <Ctrl>+<X>, para sair.

De volta ao Bash, use o comando “cat” para visualizar o conteúdo do arquivo criado no editor de texto (“cat nomedoarquivo”).

Para gerenciar o espaço no sistema de arquivos, o comando “df” exhibe informações sobre o espaço em disco utilizado e disponível em cada sistema de arquivos montado. Use “df -h”. Já o comando “du” estima o uso

de espaço em disco pelos arquivos e diretórios. Use “du -h” e compare com uma versão mais sintética, usando “du -hd1” e “du -hd1 /”.

Para visualizar o conteúdo da variável de ambiente PATH, já foi mencionado o comando “echo \$PATH”. É possível adicionar um novo diretório ao PATH temporariamente, usando “export PATH=\$PATH:/caminho/diretorio/adicionado”. Note o “.” separando o PATH atual do diretório sendo adicionado. Para tornar a alteração permanente, adicione essa linha ao final do arquivo de configuração de perfil (~/.profile). Para fazer a alteração para todos os usuários, é possível modificar o arquivo de configuração do sistema “/etc/environment”, ou adicionar um script em “/etc/profile.d”. Essa alteração para todos os usuários deve ser feita com parcimônia, pois valerá para todos os usuários atuais e futuros do sistema.

Considerações finais

Os princípios básicos do gerenciamento de arquivos são: organizar, proteger e fornecer acesso eficiente aos dados. O gerenciamento eficiente do sistema de arquivos é uma tarefa imprescindível para o sistema operacional (SO). Entender esses conceitos é crucial para qualquer profissional da área.

Conhecer bem os principais comandos relacionados também é essencial para as tarefas diárias no sistema, então não deixe de praticar!

Referências

EMC Corporation. Information Storage and Management – Version 3 (ISM, v3). **Module 3: Data Center Environment**. 2015.

NEMETH, E. *et al.* **UNIX and Linux System Administration Handbook**. 4. ed. Boston: Prentice Hall, 2010.

TANENBAUM, A. S.; BOS, H. **Sistemas operacionais modernos**. 5. ed. Porto Alegre: Bookman, 2024.

Gerenciamento de E/S

Quando pensamos em um computador executando uma aplicação, ou mesmo quando estamos desenvolvendo uma, é fácil pensar que essa aplicação interage diretamente com os dispositivos de hardware, como teclado, mouse, SSD, placa de rede, etc. Na prática, entretanto, existem intermediários que fazem essa comunicação acontecer.

No momento em que um programa precisa ler dados de um arquivo, exibir algo na tela ou enviar informações pela rede, ele gera chamadas de sistema para o kernel do sistema operacional. Na maioria dos casos,

o próprio kernel do SO não vai se comunicar diretamente com o dispositivo de E/S (periférico), mas com o módulo de E/S, o controlador do periférico e as interrupções estão envolvidas nesse processo. Por exemplo, o teclado e o mouse são lidos a partir do controlador USB e toda a interação com os discos passa pelo hardware controlador de disco.

A interface entre as aplicações e o kernel é padronizada com as chamadas de sistema e a comunicação entre o módulo de E/S e o periférico está no nível do hardware e é bem específica para cada “par” módulo de E/S – periférico. Já a comunicação entre o kernel e o controlador de E/S é uma conversa em que cada uma das partes fala um idioma diferente e ainda requer que o kernel seja um grande poliglota e exímio tradutor, pois isso garante que as aplicações possam se comunicar com os dispositivos de hardware sem precisar conhecer os detalhes técnicos de cada um deles ao traduzir as solicitações feitas pelas aplicações em comandos específicos para o controlador de hardware correspondente.

O SO fornece uma abstração (virtualização) consistente, simplificada e padronizada, permitindo que os programas acessem os recursos de hardware de forma uniforme e previsível. Essa camada de abstração simplifica o desenvolvimento de software e aumenta a portabilidade das aplicações, isto é, permite que um programa seja executado em diferentes máquinas com configurações de hardware distintas sem precisar ser modificado. Isso também garante maior flexibilidade, inclusive para os desenvolvedores, pois podem criar programas que leiam um mouse, independentemente de ser um mouse convencional, ou touchpad, conectado via cabo USB, ou bluetooth...

A exemplo do que fazem os sistemas de arquivos já estudados, que permitem que se abra, edite e salve um arquivo sempre da mesma forma, mesmo que esses arquivos possam estar em um SSD, HDD, ou cartão de memória, conectado via USB, SATA, SAS, ou via rede. O sistema operacional esconde a complexidade do hardware, apresentando-o como

um conjunto de recursos gerenciáveis e acessíveis por meio de chamadas de sistema (system calls).

Voltando à comunicação entre o kernel e o módulo de E/S, “device drivers”, ou controladores de dispositivo existem para dominar cada diferente idioma, ou melhor, os comandos específicos a serem usados com cada módulo de E/S e controlar cada periférico. Pense neles como manuais de instruções, ou dicionários, que ensinam o kernel como enviar os comandos específicos para o hardware. Os drivers são incorporados ao kernel do SO e cada dispositivo requer um driver específico, pois cada um possui suas particularidades e protocolos de comunicação e sem o driver correto o sistema operacional não consegue se comunicar com o dispositivo. É comum que o device driver seja fornecido pelo fabricante do hardware. Em outras situações, o SO terá seus drivers genéricos para determinados tipos de componente baseados em padrões. Em outras situações, será necessário um esforço de várias partes para se chegar a um ponto minimamente utilizável para o dispositivo.

Um recurso de hardware utilizado para otimizar o desempenho das operações de E/S é o acesso direto à memória, ou direct memory access (DMA). Em operações comuns, todas as transferências de dados entre a memória e os dispositivos de E/S passam pela CPU, consumindo tempo que poderia estar sendo usado para efetivamente executar o programa, além de criar gargalos desnecessários. Com o DMA, a comunicação pode ocorrer entre um módulo de E/S, como o controlador de disco e a memória principal. A CPU atua apenas na inicialização e finalização da operação, ficando liberada para as outras tarefas durante a transferência de dados. Isso aumenta significativamente o desempenho do sistema, especialmente em operações que envolvem grandes volumes de dados e é amplamente utilizado na memória secundária (Stallings, 2024).

Mais recentemente, uma variação da técnica foi implementada para viabilizar a comunicação entre o módulo de E/S e o último nível da memória cache. É o chamado acesso direto à cache, ou direct cache access

(DCA), muito utilizado após o advento de placas de rede com e sem fios com taxas de transmissão de vários Gigabits por segundo (Gbps) (Stallings, 2024).

Outra técnica utilizada em muitos dispositivos de E/S é o buffering. Um buffer é uma área de memória temporária usada para armazenar dados durante as transferências de E/S. Isso permite que os dispositivos operem em velocidades diferentes e ajuda a evitar gargalos no sistema. O uso de buffers também pode melhorar a eficiência das operações de E/S, permitindo que o SO agrupe várias solicitações pequenas em uma única transferência maior. Há circuitos de memória dedicados para os buffers em muitos controladores e periféricos, como placas de rede, HDDs e SSDs, além do uso de partes da memória principal do computador para a implementação de buffers.

Nos sistemas Unix e no Linux, o diretório `/dev` permite aos processos utilizarem os dispositivos, enquanto o diretório `/sys/devices` permite administrar e visualizar informações dos dispositivos (Ward, 2015). Os dispositivos de Entrada/Saída (E/S) podem ser classificados em dispositivos de bloco e dispositivos de caractere:

- **Dispositivos de bloco:** transferem dados em blocos de tamanho fixo, como unidades de estado sólido (SSDs), discos rígidos (HDDs) e pendrives. Ou seja, correspondem aos dispositivos de memória secundária.
- **Dispositivos de caractere:** transferem dados por meio de um caractere por vez, ou uma palavra por vez, como terminal, impressoras, vídeo, etc.

Execute `ls -l` nos diretórios mencionados e repare que o caractere inicial apresenta um `"b"` para os dispositivos de bloco e um `"c"` para os de caractere. Uma lista categorizada dos dispositivos também pode ser encontrada no arquivo `/proc/devices`. Utilize `cat /proc/devices` para visualizar. Investigue os comandos `"iostat"` e `"iotop"` para ver como

monitorar o desempenho do sistema de E/S e “udevadm” para monitorar mensagens do kernel relacionadas ao gerenciamento de E/S. Para entender melhor o esquema de endereçamento de dispositivos no Linux, consulte Ward (2015).

1 Quando um sistema se conecta com outros

A capacidade de um sistema se conectar a outros sistemas é uma característica fundamental dos computadores modernos. As primeiras redes de computadores surgiram pouco tempo após os primeiros sistemas operacionais e, atualmente, a maioria dos sistemas precisa se comunicar com outros por meio de redes. Através de redes locais (LAN), das redes de longa distância (WAN) e internet, os computadores precisam ser capazes de trocar informações e compartilhar recursos, inclusive formando sistemas distribuídos.

O acesso à rede é feito por um dispositivo de E/S, a placa de rede, ou Network Interface Card (NIC) em inglês, utilizando drivers específicos e seguindo um conjunto de protocolos (regras) de comunicação como TCP, UDP, IP e outros. Novamente, o SO fornece uma interface para as aplicações acessarem a rede, permitindo que elas enviem e recebam dados sem se preocupar com os detalhes complexos da implementação do protocolo. Os sockets fazem parte disso permitindo identificar cada processo no sistema que está enviando e recebendo pacotes por meio da rede.

Os sistemas distribuídos são conjuntos de computadores interconectados que trabalham juntos para realizar uma tarefa específica. Eles oferecem vantagens como escalabilidade, tolerância a falhas e alta disponibilidade. A computação em nuvem é um exemplo popular de sistema distribuído, em que os recursos de hardware e software são fornecidos como serviços pela internet, além de sistemas de big data e sistemas de simulação computacional para fins científicos.

O middleware é um software intermediário que facilita a comunicação entre diferentes aplicações em sistemas distribuídos. Ele é uma camada intermediária entre as aplicações e o SO de cada máquina que compõe o sistema, oferecendo serviços como roteamento de mensagens, gerenciamento de transações e segurança, permitindo que as aplicações interajam de forma transparente, independentemente da plataforma ou linguagem de programação utilizada.

Nos sistemas Unix e Linux, a exemplo do que ocorre com o gerenciamento de pacotes, há diversas ferramentas para as configurações de rede, como definição de endereço IP e características de cada interface de rede. Mas, para visualizar essas configurações e monitorar atividades de rede, há alguns comandos essenciais e comuns a praticamente todos os sistemas. O comando “ip” é um desses exemplos:

- **“ip addr”**: mostra o endereço IP e as principais configurações de cada interface.
- **“ip link”**: lista as interfaces e respectivos endereços MAC.
- **“ip -s link”**: mostra estatísticas das interfaces.
- **“ip route”**: mostra a tabela de roteamento.

Ao executar os comandos, sua interface pode aparecer como algo mais intuitivo como wifi0 e wifi1 para placas wireless Ethernet (Wi-Fi), ou eth0 e eth1 para placas Ethernet. Eventualmente, ela pode aparecer com códigos enigmáticos como enp## e wlp##, em que “en” significa Ethernet e “wl” Wireless LAN, o “p” corresponde ao barramento e o “s” ao slot (Valiante Filho, 2023).

Os protocolos da camada de aplicação costumam contar com os comandos correspondentes para serem manipulados, tais como “ssh”, para o protocolo SSH; “ftp”, para o protocolo FTP; e os comandos “nslookup”, “host” e “resolvetl”, para o protocolo DNS. Também há uma série de servidores correspondentes. O OpenSSH já vem instalado por

padrão, enquanto se precisar de um servidor web o “Apache 2” e o “NGINX” são bastante populares.

O protocolo ICMP é aproveitado em comandos como “ping”, “trace-path” e “mtr”. Já os sockets podem ser investigados com o comando “ss”. Teste “ss -tap”, “ss -tan”, “ss -uap” e “ss -uan”. O firewall mais famoso é o “nftables”, que substituiu o “iptables”, embora o comando em si ainda costume responder por meio do mais novo. É comum a utilização de versões um pouco menos complicadas de firewalls como o “ufw” e o “firewalld” (comando “firewall-cmd”). Confira também o “networkctl”, que corresponde ao comando dos serviços de rede do systemd.

2 Prática: scripts

A automação de tarefas é uma das principais vantagens dos sistemas operacionais modernos. Os scripts permitem criar sequências de comandos que são executadas automaticamente pelo interpretador de comandos (shell), por isso são chamados de “Shell Scripts”. Em sistemas Linux e muitos Unix, os scripts geralmente são escritos para o Bash (Bourne Again Shell).

Um script Bash começa obrigatoriamente com os caracteres “#!/”, que seriam lidos “hash” “bang” e acabaram virando “shebang”, seguido do caminho do interpretador a ser usado para aquele script. Então, para scripts Bash, ela deve ser “#!/bin/bash” sem a utilização das aspas.

Podemos digitar quaisquer comandos a serem executados sequencialmente em um script. Para inserir comentários, usamos “#” no começo da linha, ou no final da linha após os comandos. O que estiver após o caractere “#” será ignorado pelo interpretador de comandos. Há diversos comandos no Bash usados para controlar o fluxo de execução de um script como estruturas condicionais, laços e funções.

Dois comandos básicos muito úteis em um script são o “echo” e o “read”, respectivamente para exibir mensagens na tela e para ler dados do teclado e armazenar em uma variável, podendo também mostrar uma mensagem. Assim, um “Hello World!” em shell script utiliza o comando “echo “Hello World!””, e, para perguntarmos o nome de um usuário e armazenarmos na variável “nomeusuario”, podemos utilizar “read -p “Qual o seu nome” nomeusuario”. Para exibir essa variável, posteriormente será necessário utilizar um “\$” antes da variável, por exemplo, “\$nomeusuario”.

A seguir, é apresentado um exemplo simples de script.

```
#!/bin/bash
# Este é um comentário
clear    # o comando clear limpa a tela
echo    "Olá, mundo!"
echo    " "
read    -p "Digite seu nome: " nomeusuario
echo    " "
echo    "Olá para você também $nomeusuario!"
```

Digite o script em um editor de texto, como o nano. As extensões não são necessárias, mas se fizer questão, pode utilizar “.sh”. Para tornar o script executável, é necessário alterar suas permissões usando o comando “chmod”. Por exemplo, “chmod +x meuscript”. Para executar o script, digite “./meuscript” no terminal, sem as aspas. Isso é o equivalente a digitar o caminho completo do arquivo executável do script, por exemplo “/home/estudante/meuscript” fará o mesmo efeito.

Agora vejamos um exemplo de script um pouco mais avançado. Trata-se de um jogo para se tentar adivinhar um número aleatório gerado pelo sistema por meio da variável de ambiente \$RANDOM. Esse script contém um laço do tipo “while” para limitar a quantidade de palpites a serem feitos pelo jogador, e, dentro dele, uma estrutura decisória com “if” para se verificar se o palpite foi correto, abaixo ou acima do número. Faça uma boa análise da estrutura do script, pois ele está todo comentado. A utilização de comentários ao longo do script é uma

ótima prática, por exemplo, pois ela permite que você ou qualquer outra pessoa entenda melhor o script. A indentação do código do script também é essencial, porque ela facilita a leitura do código.

```
#!/bin/bash
#Script de um jogo de adivinhação de números
#Elaborado por Prof. Filippo Valiante Filho
#Versão 1 - Abril de 2025
clear
numero=$RANDOM # Gera um número aleatório de 0 a 32767.
tentativa=1 # Inicializa a variável da contagem de
tentativas
echo "Eu pensei em um número de 0 a 32767. Será que você
consegue adivinhá-lo em 15 tentativas?"
# 0 laço do tipo while serve para limitar a quantidade de
vezes que se poderá tentar acertar o número
# -le é menor ou igual - lower or equal (<=)
while [ $tentativa -le 15 ]
do
    read -p "Digite seu palpite nº $tentativa: " palpite
    tentativa=$(( $tentativa + 1 )) #incrementa a contagem
de tentativas
    # 0 if avalia se o palpite foi igual ao número
    # depois cada elif avalia se ficou abaixo ou acima
    if [ $palpite -eq $numero ] # -eq é igual - equal (=)
    then
        # 0 parâmetro -e no echo permite usar caracteres
de controle
        # como \n para nova linha e \t para tabulação
        echo -e "\n\tParabéns, você acertou!:\n"
        exit # Encerra o script.
    elif [ $palpite -lt $numero ] # -lt é menor - lower than (<)
    then
        echo "Seu palpite está abaixo do meu número."
        elif [ $palpite -gt $numero ] # -eq é maior - greater (>)
    then
        echo "Seu palpite está acima do meu número."
    else # 0 else executa se nenhuma condição anterior for
satisfeita.
        # Provavelmente com a digitação de letras ou outros
caracteres
        clear
```

```

        echo "Houve algum erro. Talvez você não tenha
inserido apenas números. Execute o script novamente."
        exit # Encerra o script.
    fi
done
echo "Você não acertou!:( "
echo "O número era $numero."

```

Se desejar que o script seja executado como se fosse um comando, isto é, digitando “nomedascript”, em vez de “./nomedascript”, você pode inserir o diretório dos scripts no path do sistema, ou copiar os scripts para um diretório que já esteja no path, por exemplo em /usr/local/bin ou /usr/bin.



NA PRÁTICA

Para aprofundar seus conhecimentos na elaboração de scripts, consulte meu repositório no GitHub (FILIPPOVF, 2023). Há modelos de script bastante úteis, comentados e organizados em uma ordem para ajudá-lo a entender as principais estruturas utilizadas. Para clonar o repositório, utilize o comando “git clone https://github.com/filippovf/exemplos_de_scripts”. Teste os scripts conforme o conteúdo estudado neste livro.

Considerações finais

Neste capítulo final, abordamos o gerenciamento da E/S e os comandos básicos de rede que, além de fazer parte do sistema de E/S de um computador, são essenciais para a utilização de um computador atualmente.

Finalizamos com a criação de scripts, uma ferramenta poderosa e muito utilizada para automatizar tarefas na administração de um sistema. Você pode e deve elaborar scripts para automatizar as tarefas mais utilizadas no dia a dia. Pode ser lidar com repositórios em uma

plataforma de “git” como o GitHub, compilar e testar programas, fazer backups, gerenciar usuários, atualizar o sistema, verificar logs, criar relatórios do sistema, etc. Essa é uma habilidade essencial para qualquer profissional da área.

Referências

FILIPPOVF. exemplos_de_scripts. 2023. Disponível em: https://github.com/filippovf/exemplos_de_scripts. Acesso em: 17 maio 2025.

STALLINGS, W. **Arquitetura e organização de computadores**: projetando com foco em desempenho. 11. ed. Porto Alegre: Bookman, 2024.

VALIANTE FILHO, F. **Nanocurso de Linux**. 2023. Disponível em: <https://prof.valiante.info/aulas/nanocurso-de-linux-especial>. Acesso em: 30 mar. 2025.

WARD, B. **Como o Linux Funciona**: o que todo superusuário deveria saber. São Paulo: Novatec, 2015.

Sobre o autor

Filippo Valiante Filho é doutorando em engenharia elétrica pela Escola Politécnica da Universidade de São Paulo (USP) e mestre em engenharia elétrica pela mesma instituição. Possui graduação em engenharia elétrica pela Pontifícia Universidade Católica de São Paulo (PUC-SP). Já atuou nas áreas de gestão acadêmica, telecomunicações, comunicação de dados, projetos de hardware, propriedade industrial e áudio. É professor do Centro Universitário Senac desde 2018.

